

# SONG SONG HÓA AI

Từ nguyên lý cổ điển đến thực thi  
các hệ thống Deep Learning quy mô lớn



Trần Khánh Trang

TSCH TECHNICAL  
PRESS PRESS

QUYỂN HƯỚNG DẪN CHUYÊN SÂU

**SONG SONG HÓA AI**

**TỪ NGUYÊN LÝ CỐ ĐIỂN ĐẾN THỰC**

**THI**

**CÁC HỆ THỐNG QUY MÔ LỚN**

**CHUYÊN ĐỀ CHUYÊN SÂU VỀ TÍNH TOÁN SONG SONG**

*Phân tích bản chất Tensor, Model Parallelism và AI-PCAM methodology*

**Tác giả: Trần Khánh Trang**

Email: [khanhtrang@vnpt.vn](mailto:khanhtrang@vnpt.vn)

**ẤN PHẨM CÁ NHÂN – 2026**

# LỜI MỞ ĐẦU & CẢM ƠN

Cuốn sách này là kết quả của một hành trình tìm tòi và đam mê đối với sự giao thoa giữa khoa học máy tính và trí tuệ nhân tạo.

Trong kỷ nguyên mà các mô hình ngôn ngữ lớn (LLM) đang định nghĩa lại cách chúng ta tương tác với thế giới, sức mạnh tính toán song song không còn chỉ là mười một công cụ tối ưu, mà đã trở thành nền tảng của tri thức số. Cuốn sách này được viết với mong muốn hệ thống hóa những nguyên lý song song hóa từ cốt lõi cổ điển đến những hệ thống thực thi GPU hiện đại nhất, nhằm mang lại một cái nhìn xuyên suốt cho những ai đang dấn thân vào con đường chinh phục các mô hình AI quy mô lớn.

Tác giả xin gửi lời cảm ơn chân thành đến những người thầy đã khai mở tư duy, sự hỗ trợ từ các cộng sự và cộng đồng công nghệ đã không ngừng truyền cảm hứng để tôi hoàn thiện tác phẩm này. Đặc biệt, cảm ơn sự kiên trì của chính bản thân trong những đêm miệt mài bên những dòng code và cấu trúc song song phức tạp.

Hy vọng rằng, “Song song hóa AI” sẽ trở thành một người đồng hành hữu ích cho bạn trên hành trình khám phá những giới hạn mới của công nghệ.

*Hà Nội, năm 2026*

**Trần Khánh Trang**

# TÓM TẮT

Sự bùng nổ của trí tuệ nhân tạo (AI) hiện đại, đặc biệt với các kiến trúc Transformer và mô hình ngôn ngữ lớn (LLM), đã đẩy nhu cầu tính toán vượt xa năng lực của các hệ thống đơn lẻ. Quá trình huấn luyện và triển khai các hệ thống quy mô lớn này hiện nay không thể tách rời các nguyên lý của tính toán song song và điện toán hiệu năng cao (HPC).

Báo cáo này cung cấp một nghiên cứu hệ thống về tính toán song song trong kỷ nguyên AI. Nghiên cứu bắt đầu bằng việc phân tích bản chất tensor của các phép toán trong học sâu, từ đó đi sâu vào các hình thức song song hóa then chốt: song song dữ liệu (data parallelism), song song mô hình (model parallelism), song song pipeline và song song tensor. Trên nền tảng đó, báo cáo làm rõ các thách thức kỹ thuật ở mức hệ thống, bao gồm truyền thông đồng bộ gradient, quản lý phân tầng bộ nhớ, và các bài toán đánh đổi (trade-offs) phức tạp giữa tốc độ tính toán, dung lượng bộ nhớ và động học hội tụ.

Đóng góp quan trọng nhất của báo cáo là việc đề xuất khung phân tích **AI-PCAM** – một hướng mở rộng từ mô hình PCAM truyền thống sang bối cảnh AI hiện đại. Khung AI-PCAM nhấn mạnh rằng song song hóa trong AI không chỉ là bài toán phân rã tính toán thuần túy, mà là một quy trình đồng thiết kế (co-design) chặt chẽ giữa kiến trúc hệ thống và bản chất của thuật toán học. Thông qua đó, nghiên cứu khẳng định tính toán song song cần được nhìn nhận như một nền tảng liên ngành thiết yếu, kết nối giữa học máy, hệ thống phân tán và hạ tầng tính toán hiệu năng cao trong tương lai.

**Từ khóa:** Tính toán song song, AI, GPU, Data parallelism, Tensor parallelism, AI-PCAM, High Performance Computing.

# Mục lục

|                                                                               |           |
|-------------------------------------------------------------------------------|-----------|
| <b>LỜI MỞ ĐẦU &amp; CẢM ƠN</b>                                                | <b>i</b>  |
| <b>TÓM TẮT</b>                                                                | <b>ii</b> |
| <b>Bảng thuật ngữ và ký hiệu</b>                                              | <b>1</b>  |
| <b>Tính toán song song trong AI</b>                                           | <b>3</b>  |
| 1 Giới thiệu . . . . .                                                        | 3         |
| 2 Bản chất tính toán của AI hiện đại . . . . .                                | 4         |
| 2.1 AI hiện đại như một hệ thống tính toán tensor . . . . .                   | 4         |
| 2.2 Huấn luyện AI là một quá trình lặp quy mô lớn . . . . .                   | 5         |
| 2.3 Tính toán AI bị ràng buộc bởi compute lẫn data movement . . . . .         | 5         |
| 2.4 AI hiện đại là bài toán đồng tối ưu hiệu năng và chất lượng học . . . . . | 6         |
| 3 Các hình thức song song trong AI . . . . .                                  | 6         |
| 3.1 Data Parallelism . . . . .                                                | 6         |
| 3.2 Model Parallelism . . . . .                                               | 7         |
| 3.3 Pipeline Parallelism . . . . .                                            | 8         |
| 3.4 Tensor Parallelism . . . . .                                              | 9         |
| 3.5 So sánh các hình thức song song . . . . .                                 | 9         |
| 4 Truyền thông và đồng bộ trong huấn luyện phân tán . . . . .                 | 10        |
| 4.1 Truyền thông như một nút thắt cổ chai . . . . .                           | 10        |
| 4.2 Đồng bộ gradient và all-reduce . . . . .                                  | 11        |
| 4.3 Đồng bộ và không đồng bộ . . . . .                                        | 11        |
| 4.4 Tối ưu truyền thông . . . . .                                             | 12        |

|      |                                                                        |    |
|------|------------------------------------------------------------------------|----|
| 5    | Ràng buộc bộ nhớ và phần cứng . . . . .                                | 12 |
| 5.1  | Bộ nhớ GPU như một giới hạn thực tiễn . . . . .                        | 12 |
| 5.2  | Phân tầng bộ nhớ . . . . .                                             | 12 |
| 5.3  | Interconnect và topology phần cứng . . . . .                           | 13 |
| 5.4  | Các chiến lược giảm áp lực bộ nhớ . . . . .                            | 13 |
| 6    | Các đánh đổi ở mức hệ thống . . . . .                                  | 14 |
| 6.1  | Compute và communication . . . . .                                     | 14 |
| 6.2  | Batch size và convergence . . . . .                                    | 14 |
| 6.3  | Bộ nhớ và khả năng mở rộng . . . . .                                   | 15 |
| 6.4  | Throughput và chất lượng học . . . . .                                 | 15 |
| 7    | Framework và hệ thống AI phân tán hiện đại . . . . .                   | 15 |
| 7.1  | Từ PyTorch DistributedDataParallel đến các primitive của MPI . . . . . | 17 |
| 8    | Góc nhìn AI-PCAM . . . . .                                             | 20 |
| 8.1  | Tổng quan các hướng tiếp cận song song trong AI hiện đại . . . . .     | 20 |
| 8.2  | Khoảng trống nghiên cứu . . . . .                                      | 21 |
| 8.3  | Nhắc lại mô hình PCAM . . . . .                                        | 21 |
| 8.4  | Từ PCAM cổ điển đến AI-PCAM . . . . .                                  | 21 |
| 8.5  | Không gian quyết định trong AI-PCAM . . . . .                          | 22 |
| 8.6  | Minh họa trực quan AI-PCAM . . . . .                                   | 23 |
| 8.7  | Nhận xét tổng hợp . . . . .                                            | 25 |
| 9    | Góc nhìn lập trình: Liên hệ MPI và OpenMP . . . . .                    | 26 |
| 9.1  | Data Parallelism và mô hình SPMD trong MPI . . . . .                   | 26 |
| 9.2  | Song song trong bộ nhớ chia sẻ và OpenMP . . . . .                     | 26 |
| 9.3  | Pipeline Parallelism và song song theo tác vụ . . . . .                | 27 |
| 9.4  | Từ MPI/OpenMP đến các framework AI hiện đại . . . . .                  | 27 |
| 9.5  | Góc nhìn của lập trình viên . . . . .                                  | 27 |
| 10   | Case Study: Song song hóa trong Transformer và LLM . . . . .           | 28 |
| 10.1 | Cấu trúc tính toán của Transformer . . . . .                           | 28 |
| 10.2 | Áp dụng data parallelism trong LLM . . . . .                           | 28 |

|          |                                                                         |           |
|----------|-------------------------------------------------------------------------|-----------|
| 10.3     | Áp dụng tensor parallelism . . . . .                                    | 28        |
| 10.4     | Kết hợp pipeline parallelism . . . . .                                  | 28        |
| 10.5     | Hybrid parallelism trong thực tế . . . . .                              | 29        |
| 11       | Thảo luận và hướng nghiên cứu . . . . .                                 | 29        |
| 11.1     | LLM scaling laws và các giới hạn truyền thống . . . . .                 | 30        |
| 12       | Kết luận . . . . .                                                      | 31        |
| <b>A</b> | <b>Nguồn tài liệu và hình thành báo cáo</b>                             | <b>32</b> |
| A.0.1    | Câu hỏi khởi đầu . . . . .                                              | 32        |
| A.0.2    | Tài liệu nền tảng về song song hóa . . . . .                            | 32        |
| A.0.3    | Các tài liệu về huấn luyện phân tán trong AI . . . . .                  | 32        |
| A.0.4    | Hình thành góc nhìn AI-PCAM . . . . .                                   | 33        |
| <b>B</b> | <b>Minh họa lập trình song song</b>                                     | <b>34</b> |
| B.0.1    | OpenMP: Song song hóa vòng lặp . . . . .                                | 34        |
| B.0.2    | MPI: Tính tổng phân tán . . . . .                                       | 35        |
| B.0.3    | MPI Allreduce: Đồng bộ gradient . . . . .                               | 36        |
| B.0.4    | PyTorch: Data Parallel . . . . .                                        | 37        |
| B.0.5    | Nhận xét tổng hợp . . . . .                                             | 37        |
| <b>C</b> | <b>Ứng dụng AI phổ biến tiêu biểu</b>                                   | <b>39</b> |
| C.0.1    | Nhân ma trận với kỹ thuật Tiling . . . . .                              | 39        |
| C.0.2    | Phát hiện sâu bệnh trên lá táo với YOLO . . . . .                       | 40        |
| C.0.3    | Dự báo lũ cho Đồng bằng sông Cửu Long . . . . .                         | 40        |
| C.0.4    | Nhận xét tổng hợp . . . . .                                             | 41        |
| <b>D</b> | <b>Minh họa thực nghiệm trên GPU và liên hệ với tính toán song song</b> | <b>42</b> |
| D.0.1    | Môi trường thực nghiệm . . . . .                                        | 42        |
| D.0.2    | Demo 1: Phân loại bệnh lá táo trên GPU . . . . .                        | 42        |
| D.0.3    | Demo 2: Dự báo mưa . . . . .                                            | 43        |
| D.0.4    | Liên hệ với các nguyên lý song song . . . . .                           | 44        |

**TÀI LIỆU THAM KHẢO**

**45**

# Bảng thuật ngữ và ký hiệu

| Thuật ngữ            | Ý nghĩa                                                                  |
|----------------------|--------------------------------------------------------------------------|
| Data Parallelism     | Chia dữ liệu và sao chép mô hình trên nhiều thiết bị để xử lý song song  |
| Model Parallelism    | Chia mô hình thành nhiều phần và phân bố trên nhiều thiết bị             |
| Tensor Parallelism   | Chia tensor hoặc phép toán lớn thành các phần nhỏ hơn để xử lý song song |
| Pipeline Parallelism | Chia mô hình thành các stage và xử lý theo dạng pipeline                 |
| All-Reduce           | Cơ chế tổng hợp và phân phối gradient giữa các worker                    |
| VRAM                 | Bộ nhớ của GPU dùng để lưu trữ tensor và mô hình                         |
| Throughput           | Số lượng mẫu xử lý trên một đơn vị thời gian                             |
| Convergence          | Quá trình hội tụ của thuật toán học trong quá trình huấn luyện           |

Bảng 1: Các thuật ngữ chính sử dụng trong báo cáo

# Danh sách hình vẽ

|    |                                                                                                                                                                                                                   |    |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1  | Data Parallel: mỗi GPU xử lý dữ liệu riêng và đồng bộ gradient. . . . .                                                                                                                                           | 7  |
| 2  | Model Parallel: phân chia mô hình theo chiều sâu giữa các GPU. . . . .                                                                                                                                            | 8  |
| 3  | Pipeline Parallel: chia mô hình thành các stage và xử lý theo pipeline. . . . .                                                                                                                                   | 8  |
| 4  | Tensor Parallel: phân chia tensor hoặc ma trận trọng số giữa nhiều GPU để tính song song. . . . .                                                                                                                 | 9  |
| 5  | Cơ chế đồng bộ gradient bằng all-reduce trong huấn luyện phân tán đồng bộ, trong đó gradient cục bộ từ các GPU được tổng hợp và phân phối lại dưới dạng trung bình để đảm bảo tính nhất quán của mô hình. . . . . | 11 |
| 6  | Phân tầng bộ nhớ và liên kết giữa các GPU trong hệ AI hiện đại. . . . .                                                                                                                                           | 13 |
| 7  | Đánh đổi giữa chi phí tính toán và chi phí truyền thông khi mở rộng số lượng GPU. . . . .                                                                                                                         | 14 |
| 8  | Minh họa đánh đổi giữa kích thước batch lớn và tốc độ/chất lượng hội tụ. . . . .                                                                                                                                  | 15 |
| 9  | Mô hình song song GPU theo CUDA: Grid gồm nhiều Block, mỗi Block gồm nhiều Thread; các thread được nhóm thành warp để thực thi song song trên GPU. . . . .                                                        | 24 |
| 10 | Phân tầng bộ nhớ GPU: register (nhANH NHẤT), shared memory và global memory (VRAM), với sự đánh đổi giữa tốc độ truy cập và dung lượng lưu trữ. . . . .                                                           | 24 |
| 11 | Khung AI-PCAM cho hệ AI hiện đại. . . . .                                                                                                                                                                         | 25 |

# Tính toán song song trong AI

## 1 Giới thiệu

Sự phát triển nhanh chóng của trí tuệ nhân tạo hiện đại, đặc biệt là học sâu (Deep Learning), đã làm thay đổi căn bản quy mô và bản chất của nhu cầu tính toán trong các hệ thống phần mềm thông minh. Nếu như trong giai đoạn đầu của học máy, các mô hình truyền thống như hồi quy logistic, SVM, cây quyết định hay các mạng nơ-ron nông có thể được huấn luyện trên một máy tính cá nhân hoặc một máy chủ đơn lẻ, thì ngày nay nhiều mô hình tiên tiến đã đạt tới quy mô hàng tỷ đến hàng nghìn tỷ tham số. Quy mô đó kéo theo chi phí tính toán, lưu trữ tham số, truy cập bộ nhớ, truyền thông giữa các thiết bị và chi phí năng lượng tăng mạnh.

Trong bối cảnh này, tính toán song song (parallel computing) không còn là một công cụ tối ưu hiệu năng mang tính bổ trợ, mà trở thành điều kiện nền tảng để các hệ thống AI hiện đại có thể được huấn luyện, triển khai và mở rộng. Một mô hình ngôn ngữ lớn, một hệ thống khuyến nghị quy mô công nghiệp, hay một mô hình thị giác đa phương thức hiện đại hầu như không thể vận hành hiệu quả nếu chỉ dựa trên cấu trúc thực thi tuần tự. Thay vào đó, chúng đòi hỏi sự phối hợp giữa nhiều bộ xử lý, nhiều GPU, nhiều tầng bộ nhớ và nhiều cơ chế đồng bộ, từ đó hình thành một không gian thiết kế hệ thống phức tạp.

Điều quan trọng là song song hóa trong AI không đơn thuần là “chia việc cho nhiều thiết bị làm cùng lúc”. Trong thực tế, mỗi quyết định song song hóa đều tác động đồng thời đến nhiều lớp của hệ thống. Ở lớp mô hình, việc phân tách dữ liệu hay phân mảnh tensor có thể làm thay đổi cách mô hình được học. Ở lớp tối ưu hóa, batch size lớn hơn có thể làm thay đổi nhiều gradient và động học hội tụ. Ở lớp hệ thống, số lượng GPU nhiều hơn có thể làm chi phí truyền thông tăng mạnh. Ở lớp phần cứng, cấu trúc liên kết như PCIe, NVLink, InfiniBand hoặc cấu trúc liên nút sẽ quyết định việc mở rộng có đạt hiệu quả hay không. Do đó, song song hóa trong AI hiện đại không chỉ là vấn đề tối ưu hiệu năng, mà là bài toán đồng thiết kế giữa tính toán, dữ liệu, mô hình học và hạ tầng phần cứng.

Từ góc độ học thuật, có thể xem bài toán song song hóa trong AI hiện đại là điểm giao thoa của ba lĩnh vực lớn. Thứ nhất là học máy, nơi trọng tâm nằm ở khả năng học biểu diễn, tối ưu hóa gradient, hội tụ và tổng quát hóa. Thứ hai là hệ thống phân tán, nơi trọng tâm nằm ở lập lịch, chia tải, truyền thông, đồng bộ và độ tin cậy. Thứ ba là tính toán hiệu năng cao (High Performance

Computing – HPC), nơi các nguyên lý thiết kế song song, ánh xạ tác vụ và tối ưu tài nguyên đã được nghiên cứu từ lâu. Tuy nhiên, điều làm cho AI hiện đại khác biệt là ba lớp này không thể tách rời; một quyết định ở lớp hệ thống có thể làm thay đổi hành vi học, và ngược lại.

Chương này được xây dựng nhằm đạt ba mục tiêu chính. Thứ nhất, phân tích bản chất tính toán của AI hiện đại và lý do vì sao các hệ AI quy mô lớn phải dựa trên các chiến lược song song hóa. Thứ hai, hệ thống hóa các hình thức song song chủ đạo trong AI, bao gồm data parallelism, model parallelism, pipeline parallelism và tensor parallelism, đồng thời phân tích các đánh đổi về truyền thông, bộ nhớ, khả năng mở rộng và chất lượng học. Thứ ba, trên cơ sở tổng hợp các nguyên lý đó, đề xuất khung AI-PCAM như một cách diễn giải mở rộng của tư duy thiết kế song song trong bối cảnh AI hiện đại.

Báo cáo này được xây dựng dựa trên các nguyên lý cốt lõi của môn học Tính toán song song nâng cao. Các khái niệm như phân rã bài toán, đồng bộ hóa, truyền thông và ánh xạ được mở rộng và áp dụng trong AI hiện đại, nơi các hệ thống GPU phân tán đóng vai trò trung tâm.

## 2 Bản chất tính toán của AI hiện đại

Từ góc nhìn hệ thống, bản chất tính toán của AI hiện đại có thể được phân tích qua ba khía cạnh chính: tính toán tensor, tính lặp của quá trình huấn luyện, và mối quan hệ giữa compute với data movement.

### 2.1 AI hiện đại như một hệ thống tính toán tensor

Ở mức cốt lõi, phần lớn các hệ AI hiện đại được xây dựng xung quanh các phép toán tensor. Tensor có thể hiểu là sự tổng quát hóa của số vô hướng, véc-tơ và ma trận lên nhiều chiều hơn. Trong học sâu, đầu vào thường được biểu diễn dưới dạng tensor, trọng số của mạng cũng được lưu ở dạng tensor, còn các trạng thái trung gian (activation) phát sinh trong quá trình lan truyền thuận và lan truyền ngược cũng đều là tensor.

Về mặt tính toán, nhiều phép toán quan trọng trong AI có thể quy về một số nhóm chính: phép nhân ma trận, phép cộng phần tử, phép chuẩn hóa, phép kích hoạt phi tuyến, phép tích chập và phép attention. Trong số đó, phép nhân ma trận quy mô lớn chiếm vị trí đặc biệt quan trọng. Đối với các mô hình transformer, phần lớn chi phí tính toán nằm ở các phép nhân ma trận trong lớp attention và các khối feed-forward. Điều này giải thích vì sao các bộ xử lý chuyên cho AI, đặc biệt là GPU và TPU, được thiết kế để tối ưu hóa mạnh cho nhóm phép toán dạng GEMM.

Bản chất tensor hóa của tính toán AI có hai hệ quả lớn. Thứ nhất, nhiều phép toán có cấu trúc dữ liệu đều đặn, rất phù hợp cho vectorization và parallelization. Thứ hai, hiệu năng của hệ thống phụ thuộc mạnh vào cách tensor được bố trí trong bộ nhớ, di chuyển giữa các tầng bộ nhớ và

phân chia giữa các thiết bị. Nói cách khác, trong AI hiện đại, biểu diễn toán học và biểu diễn hệ thống của dữ liệu không thể tách rời.

## 2.2 Huấn luyện AI là một quá trình lặp quy mô lớn

Huấn luyện mô hình AI không phải là một phép tính một lần, mà là một quy trình lặp nhiều bước. Với một mini-batch dữ liệu, hệ thống thực hiện lan truyền thuận để tạo đầu ra, tính hàm mất mát, lan truyền ngược để tính gradient và cuối cùng cập nhật tham số thông qua thuật toán tối ưu. Chu trình này lặp lại hàng nghìn, hàng chục nghìn hoặc hàng triệu bước.

Tính lặp này tạo ra một đặc điểm khác biệt của AI so với nhiều bài toán HPC truyền thống. Trong một số bài toán khoa học tính toán, việc chia bài toán song song chủ yếu nhằm rút ngắn thời gian thực hiện của một phép tính lớn. Trong AI, bài toán song song hóa diễn ra lặp lại trong mỗi iteration. Vì vậy, chi phí nhỏ ở một bước đơn lẻ có thể tích lũy thành chi phí khổng lồ trên toàn bộ thời gian huấn luyện. Chẳng hạn, chỉ cần mỗi bước đồng bộ gradient chậm hơn vài mili giây, tổng chi phí phát sinh trên hàng trăm nghìn bước có thể rất lớn.

Tính lặp cũng kéo theo một yêu cầu quan trọng: hệ thống song song hóa phải không chỉ nhanh, mà còn ổn định trong suốt quá trình học. Nếu một cấu hình phân tán giúp tăng throughput nhưng làm gradient bị sai lệch quá mức hoặc làm động học hội tụ trở nên kém ổn định, thì lợi ích hiệu năng ngắn hạn có thể không bù lại được chi phí học kém hiệu quả.

## 2.3 Tính toán AI bị ràng buộc bởi compute lẫn data movement

Một ngộ nhận phổ biến là các mô hình AI hiện đại chỉ bị giới hạn bởi năng lực tính toán thuần túy, ví dụ số FLOPs hoặc số nhân CUDA. Trên thực tế, trong rất nhiều trường hợp, hệ thống bị ràng buộc không chỉ bởi compute mà còn bởi khả năng di chuyển dữ liệu (data movement). Mỗi bước lan truyền thuận và lan truyền ngược đều cần truy xuất trọng số, activation, gradient và nhiều trạng thái optimizer từ bộ nhớ. Nếu các đối tượng này không nằm đủ gần bộ xử lý, thời gian chờ bộ nhớ có thể trở thành nút thắt lớn.

Khi mở rộng sang nhiều GPU, bài toán còn trở nên phức tạp hơn. Không chỉ dữ liệu phải di chuyển trong một GPU giữa global memory, shared memory và register, mà còn phải di chuyển giữa các GPU hoặc giữa các node. Từ đây nảy sinh vấn đề băng thông, độ trễ, topology liên kết và sự phối hợp giữa truyền thông với tính toán. Do đó, một hệ thống song song hiệu quả không chỉ tối đa hóa compute, mà còn phải tối thiểu hóa data movement và chồng lấp hai quá trình này ở mức cao nhất có thể.

## 2.4 AI hiện đại là bài toán đồng tối ưu hiệu năng và chất lượng học

Điểm khác biệt quan trọng giữa song song hóa trong AI và nhiều ứng dụng truyền thống là: tăng tốc thực thi không phải mục tiêu duy nhất. Một chiến lược tốt phải đồng thời đảm bảo chất lượng học. Ví dụ, tăng batch size tổng bằng cách sử dụng nhiều GPU có thể giúp throughput tăng mạnh, nhưng cũng có thể làm giảm nhiễu gradient và tác động đến nghiệm tối ưu mà mô hình tìm thấy. Tương tự, truyền thông không đồng bộ có thể cải thiện mức sử dụng tài nguyên, nhưng lại làm tham số bị cập nhật dựa trên gradient lỗi thời.

Do đó, trong AI hiện đại, hiệu năng không thể được đánh giá chỉ bằng thời gian chạy hoặc số mẫu trên giây. Cần xem xét đồng thời các tiêu chí như tốc độ hội tụ, độ chính xác cuối cùng, khả năng tổng quát hóa và tính ổn định của quá trình huấn luyện. Đây chính là lý do khiến thiết kế hệ thống AI phân tán trở thành một bài toán nhiều mục tiêu.

## 3 Các hình thức song song trong AI

Từ các đặc điểm tính toán trên, các chiến lược song song hóa được phát triển nhằm khai thác cấu trúc tensor và tính lặp của quá trình huấn luyện.

### 3.1 Data Parallelism

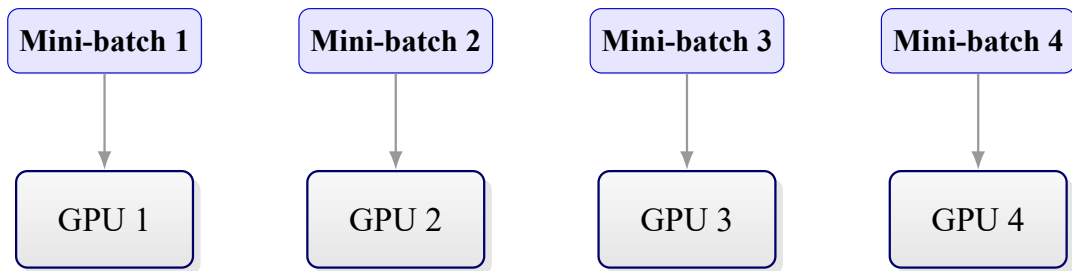
Data parallelism là chiến lược song song hóa được sử dụng rộng rãi nhất trong thực tiễn huấn luyện mô hình AI. Ý tưởng cơ bản là sao chép cùng một mô hình lên nhiều GPU hoặc nhiều node, sau đó chia dữ liệu thành các mini-batch con để mỗi thiết bị xử lý một phần. Sau bước lan truyền ngược, các gradient cục bộ được đồng bộ để toàn bộ thiết bị cập nhật cùng một trạng thái tham số.

Ưu điểm lớn nhất của data parallelism là tính trực quan và khả năng tích hợp tốt với nhiều framework hiện đại. Đối với các mô hình có thể nằm trọn trong bộ nhớ của một GPU, data parallelism thường là lựa chọn đầu tiên vì nó cho phép tận dụng nhiều thiết bị mà không cần thay đổi quá sâu cấu trúc mô hình. Ngoài ra, nó tương thích khá tốt với logic của stochastic gradient descent, vì mỗi mini-batch cục bộ thực chất chỉ là một phần của batch tổng.

Tuy nhiên, khi quy mô mô hình hoặc số lượng thiết bị tăng lên, data parallelism bộc lộ một số giới hạn. Trước hết, mô hình phải được lưu đầy đủ trên mỗi thiết bị, điều này làm giới hạn khả năng áp dụng cho mô hình rất lớn. Thứ hai, chi phí đồng bộ gradient tăng cùng số lượng tham số và số lượng thiết bị. Thứ ba, để tận dụng nhiều GPU, người ta thường tăng batch size tổng, kéo theo các hệ quả lên động học học tập.

Data parallelism trong AI có thể được xem là một dạng mở rộng của mô hình SPMD trong lập trình MPI, trong đó mỗi tiến trình xử lý một phần dữ liệu độc lập và thực hiện bước đồng bộ hóa

thông qua các primitive như `MPI_Allreduce`. Cơ chế đồng bộ gradient trong Deep Learning tương đương với việc sử dụng MPI collective communication trong các bài toán HPC truyền thống.



*Mỗi GPU xử lý một mini-batch độc lập, sau đó đồng bộ gradient qua all-reduce.*

**Hình 1:** Data Parallel: mỗi GPU xử lý dữ liệu riêng và đồng bộ gradient.

Hình ?? mô tả cách dữ liệu được chia cho nhiều GPU, trong khi cấu trúc mô hình vẫn được giữ giống nhau trên mọi thiết bị. Đây là hình thức song song phù hợp với nhiều hệ huấn luyện tiêu chuẩn, nhưng hiệu quả của nó phụ thuộc mạnh vào chất lượng cơ chế all-reduce và băng thông liên kết giữa các GPU.

### 3.2 Model Parallelism

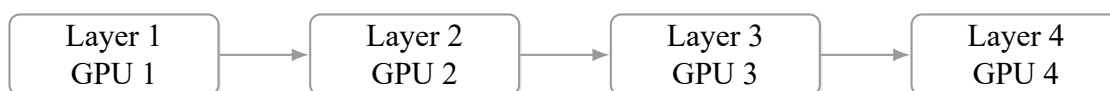
Model parallelism xuất hiện khi mô hình quá lớn để có thể chứa nguyên vẹn trên một thiết bị đơn lẻ. Trong trường hợp này, mô hình được chia thành nhiều phần, ví dụ theo tầng, theo khối hoặc theo module chức năng, rồi đặt trên các GPU khác nhau. Dữ liệu đầu vào phải đi qua các phần đó theo trình tự của đồ thị tính toán.

Điểm mạnh của model parallelism là giải quyết trực tiếp ràng buộc bộ nhớ, đặc biệt quan trọng đối với transformer quy mô lớn. Ngoài ra, nó cho phép mở rộng mô hình vượt quá kích thước VRAM của từng thiết bị. Tuy nhiên, nó làm gia tăng truyền thông giữa các thiết bị vì activation phải được chuyển từ phần mô hình này sang phần mô hình khác, sau đó gradient lại phải đi ngược trong quá trình backward pass.

Một thách thức quan trọng của model parallelism là cân bằng tải. Nếu các phần mô hình không đồng đều về chi phí tính toán hoặc kích thước activation, sẽ xảy ra hiện tượng một số GPU làm việc nặng hơn số còn lại. Khi đó, thời gian thực thi toàn hệ thống bị chi phối bởi phần chậm nhất.

Ở cấp độ intra-node, việc sử dụng nhiều GPU hoặc nhiều luồng CPU có thể được liên hệ với mô hình shared-memory trong OpenMP, trong đó các luồng cùng truy cập một không gian bộ nhớ

chung và được đồng bộ thông qua các cơ chế barrier hoặc critical section.



*Mô hình được chia theo chiều sâu, mỗi GPU xử lý một phần của mạng nơ-ron và truyền activation tuần tự.*

**Hình 2:** Model Parallel: phân chia mô hình theo chiều sâu giữa các GPU.

Hình ?? minh họa cách các tầng của mô hình được phân bổ lên nhiều GPU. Đây là một dạng song song hóa hữu ích nhưng đòi hỏi thiết kế cẩn thận hơn so với data parallelism.

### 3.3 Pipeline Parallelism

Pipeline parallelism có thể được xem là sự phát triển tự nhiên của model parallelism khi người ta cố gắng tăng mức sử dụng tài nguyên và giảm thời gian nhàn rỗi của các GPU. Trong phương pháp này, mô hình được chia thành nhiều stage, mỗi stage nằm trên một GPU hoặc một nhóm GPU. Dữ liệu không đi qua toàn bộ pipeline theo cách tuần tự đơn giản, mà được tách thành nhiều micro-batch để các stage có thể làm việc chồng lấp theo thời gian.

Ưu điểm lớn nhất của pipeline parallelism là nâng cao mức độ sử dụng thiết bị. Khi pipeline được làm đầy, mỗi stage có thể đồng thời xử lý một micro-batch khác nhau. Tuy nhiên, pipeline parallelism tạo ra hiện tượng pipeline bubble, tức khoảng thời gian đầu và cuối khi một số stage chưa có việc hoặc đã hoàn thành sớm hơn các stage khác. Mức độ bubble phụ thuộc vào số lượng stage, số lượng micro-batch và cách lên lịch giữa forward và backward pass.

Pipeline parallelism có thể được diễn giải tương tự mô hình task parallelism, trong đó các tác vụ được phân chia theo từng giai đoạn xử lý và thực thi chồng lấp theo thời gian.



*Các mini-batch được đưa vào pipeline theo dạng “dây chuyền”, giúp tăng throughput nhưng cần xử lý bubble pipeline.*

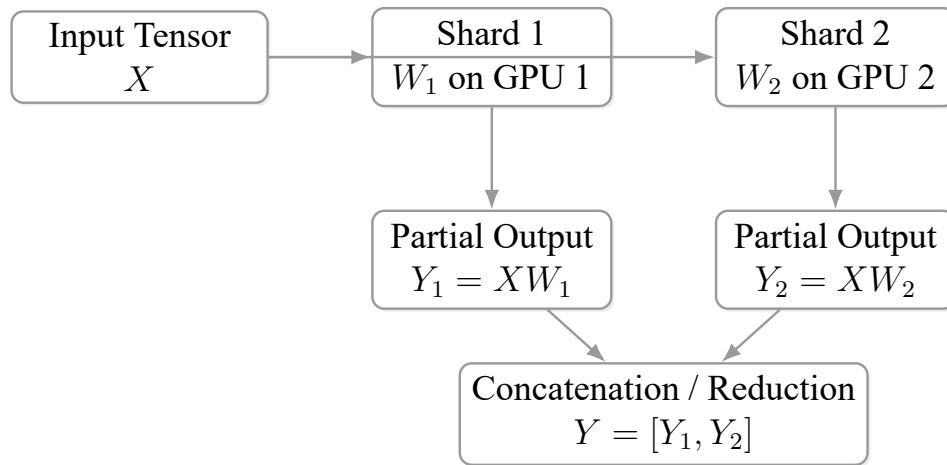
**Hình 3:** Pipeline Parallel: chia mô hình thành các stage và xử lý theo pipeline.

Hình ?? cho thấy cách các micro-batch dịch chuyển qua các stage. Trong thực tế, việc chọn số micro-batch là một quyết định hệ thống quan trọng: quá ít thì bubble lớn, quá nhiều thì overhead quản lý và bộ nhớ tăng.

### 3.4 Tensor Parallelism

Tensor parallelism chia trực tiếp các tensor hoặc phép toán tensor lớn thành nhiều phần nhỏ hơn, phân bố lên nhiều GPU và phối hợp để tạo ra kết quả cuối cùng. Trong transformer, tensor parallelism thường được dùng trong các phép nhân ma trận lớn của lớp attention và feed-forward network. Thay vì gán nguyên một tầng lên một GPU, người ta chia ma trận trọng số hoặc activation thành nhiều khối.

Điểm mạnh nổi bật của tensor parallelism là có thể khai thác song song ở mức hạt mịn hơn model parallelism. Đây là một công cụ đặc biệt mạnh đối với mô hình cực lớn, nơi các phép toán bên trong từng tầng cũng đã đủ lớn để cần được chia nhỏ. Tuy vậy, cái giá phải trả là truyền thông liên tục hơn trong từng bước tính toán. Kết quả trung gian từ các phần tensor khác nhau thường phải được ghép lại, cộng lại hoặc phát lại, khiến tần suất truyền thông tăng lên.



*Tensor parallel chia một phép toán lớn hoặc một ma trận trọng số thành nhiều phần nhỏ trên nhiều GPU, phù hợp với mô hình rất lớn khi một thiết bị không đủ bộ nhớ.*

**Hình 4:** Tensor Parallel: phân chia tensor hoặc ma trận trọng số giữa nhiều GPU để tính song song.

Hình ?? minh họa một phép nhân tensor lớn được chia thành nhiều khối nhỏ hơn. Trong các hệ huấn luyện LLM hiện đại, tensor parallelism thường được kết hợp với data parallelism và pipeline parallelism chứ hiếm khi được dùng đơn lẻ.

### 3.5 So sánh các hình thức song song

Để có cái nhìn hệ thống hơn, Bảng 2 tóm tắt một số tiêu chí so sánh giữa bốn hình thức song song chính.

Bảng 2: So sánh các hình thức song song trong AI hiện đại

| Hình thức            | Ý tưởng chính                                            | Ưu điểm                                               | Hạn chế                                                         | Trường hợp phù hợp                                 |
|----------------------|----------------------------------------------------------|-------------------------------------------------------|-----------------------------------------------------------------|----------------------------------------------------|
| Data Parallelism     | Chia dữ liệu, sao chép mô hình trên nhiều thiết bị       | Dễ triển khai, tương thích tốt với framework phổ biến | Đồng bộ gradient tốn kém, mô hình phải vừa bộ nhớ mỗi GPU       | Mô hình vừa VRAM, cần scale số mẫu/giấy            |
| Model Parallelism    | Chia mô hình thành nhiều phần trên các GPU khác nhau     | Vượt giới hạn bộ nhớ của một GPU                      | Activation/gradient phải truyền giữa các phần; cân bằng tải khó | Mô hình lớn hơn bộ nhớ một GPU                     |
| Pipeline Parallelism | Chia mô hình thành stage, đưa micro-batch qua pipeline   | Tăng mức sử dụng GPU, hỗ trợ mô hình lớn              | Xuất hiện pipeline bubble, lịch thực thi phức tạp               | Mô hình sâu, nhiều stage, huấn luyện dài           |
| Tensor Parallelism   | Chia trực tiếp tensor/phép toán lớn thành nhiều phần nhỏ | Khai thác song song hạt mịn, hiệu quả cho LLM         | Truyền thông thường xuyên, triển khai khó hơn                   | Ma trận lớn, attention/feed-forward quy mô rất lớn |

Từ bảng trên có thể thấy không tồn tại một chiến lược song song hóa tối ưu cho mọi bài toán. Trong thực tế, các hệ thống hiện đại thường kết hợp nhiều chiến lược để đạt được sự cân bằng giữa bộ nhớ, hiệu năng tính toán và chi phí truyền thông.

## 4 Truyền thông và đồng bộ trong huấn luyện phân tán

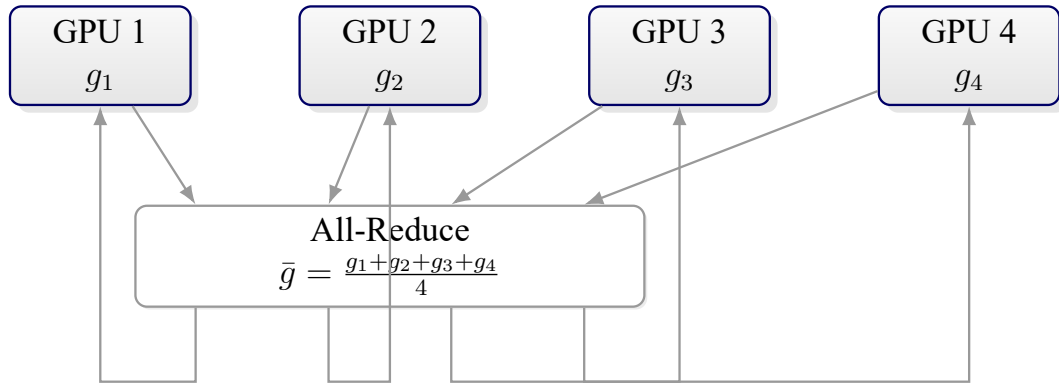
### 4.1 Truyền thông như một nút thắt cổ chai

Trong nhiều bài toán song song truyền thống, thêm bộ xử lý thường mang lại tăng tốc đáng kể nếu bài toán đủ lớn và có độ độc lập cao. Trong AI hiện đại, đặc biệt là huấn luyện phân tán, việc thêm GPU không phải lúc nào cũng dẫn đến tăng tốc gần tuyến tính. Nguyên nhân cốt lõi là truyền thông giữa các thiết bị bắt đầu chiếm tỷ trọng lớn.

Mỗi bước huấn luyện phân tán đồng bộ đều cần trao đổi một lượng dữ liệu đáng kể, bao gồm gradient, activation hoặc tham số tùy chiến lược song song hóa. Khi quy mô mô hình tăng, số tham số cũng tăng, kéo theo kích thước gradient tăng. Khi số thiết bị tăng, số luồng giao tiếp và yêu cầu đồng bộ cũng tăng. Nếu phần cứng hoặc topology mạng không đủ tốt, hệ thống nhanh chóng rơi vào trạng thái communication-bound.

## 4.2 Đồng bộ gradient và all-reduce

Trong data parallelism, sau khi mỗi GPU tính gradient cục bộ trên mini-batch riêng, hệ thống cần gộp các gradient đó lại để mọi thiết bị cập nhật cùng một mô hình. All-reduce là một primitive tiêu biểu cho mục tiêu này. Về trực giác, all-reduce thực hiện hai nhiệm vụ đồng thời: tổng hợp dữ liệu từ mọi thiết bị và phát ngược kết quả đã tổng hợp đó trở lại tất cả các thiết bị.



*Mỗi GPU tính gradient cục bộ, sau đó hệ thống gộp và phát lại gradient trung bình cho toàn bộ thiết bị.*

**Hình 5:** Cơ chế đồng bộ gradient bằng all-reduce trong huấn luyện phân tán đồng bộ, trong đó gradient cục bộ từ các GPU được tổng hợp và phân phối lại dưới dạng trung bình để đảm bảo tính nhất quán của mô hình.

Hình 5 minh họa cơ chế này ở mức khái niệm. Mỗi GPU gửi gradient cục bộ của mình vào quá trình tổng hợp, sau đó nhận lại gradient trung bình. Điểm đáng chú ý là all-reduce không chỉ là một thao tác phụ; nó chính là bước đảm bảo tính nhất quán của huấn luyện phân tán đồng bộ.

## 4.3 Đồng bộ và không đồng bộ

Có hai triết lý lớn trong phối hợp huấn luyện phân tán: đồng bộ và không đồng bộ. Trong chế độ đồng bộ, mọi GPU phải hoàn thành bước tính cục bộ trước khi đồng bộ gradient và bước sang iteration tiếp theo. Cách tiếp cận này đơn giản hơn về mặt lý thuyết tối ưu hóa và đảm bảo mọi thiết bị làm việc trên cùng một phiên bản tham số. Tuy nhiên, hiệu năng hệ thống bị ràng buộc bởi thiết bị chậm nhất, thường gọi là straggler effect.

Trong chế độ không đồng bộ, các thiết bị có thể cập nhật tham số mà không cần chờ nhau hoàn toàn. Cách tiếp cận này có thể nâng cao mức sử dụng tài nguyên và giảm thời gian chờ, nhưng làm phát sinh vấn đề stale gradients. Gradient được tính trên tham số cũ có thể làm quá trình tối ưu hóa kém ổn định hơn, nhất là khi độ trễ giữa các cập nhật lớn.

## 4.4 Tối ưu truyền thông

Để giảm chi phí truyền thông, nhiều hướng tối ưu đã được phát triển. Một là nén gradient hoặc truyền gradient ở độ chính xác thấp hơn. Hai là chồng lấp tính toán với truyền thông, sao cho một phần gradient được gửi đi khi các phần khác của backward pass vẫn đang diễn ra. Ba là tối ưu topology truyền thông, ví dụ ring all-reduce, tree-based reduction hoặc hierarchical reduction. Bốn là giảm tần suất đồng bộ bằng gradient accumulation hoặc thay đổi batch strategy.

Nhìn từ góc độ hệ thống, truyền thông không chỉ là một bước hậu xử lý sau tính toán, mà là một cấu phần nội tại của thiết kế song song hóa trong AI.

## 5 Ràng buộc bộ nhớ và phần cứng

Bên cạnh truyền thông, bộ nhớ và kiến trúc phần cứng là các yếu tố quan trọng quyết định hiệu năng của hệ thống AI.

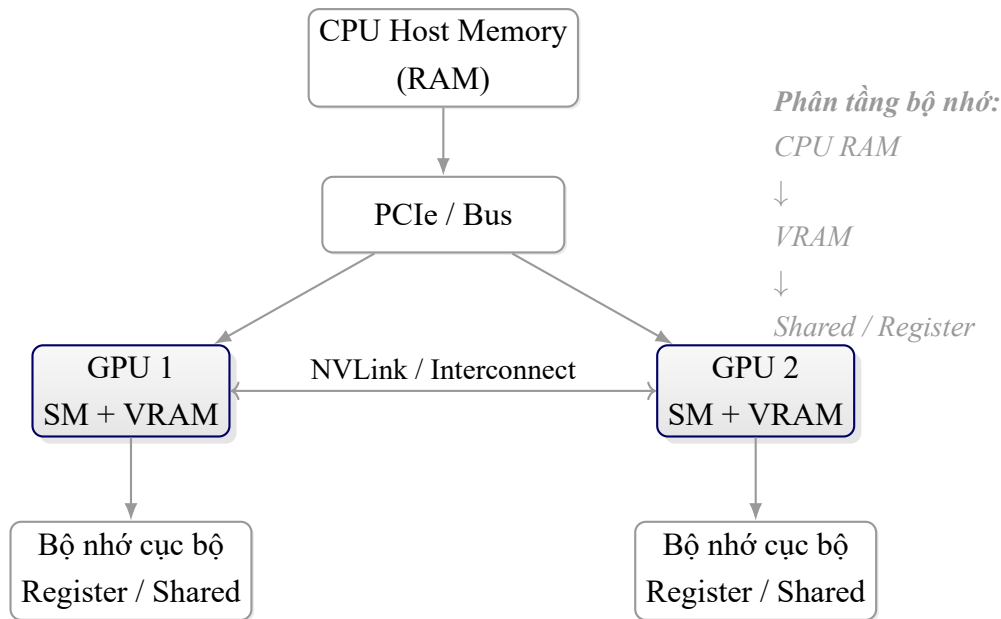
### 5.1 Bộ nhớ GPU như một giới hạn thực tiễn

Một mô hình lớn không chỉ chiếm bộ nhớ ở phần tham số. Trong quá trình huấn luyện, activation ở các tầng trung gian phải được giữ lại để phục vụ lan truyền ngược. Ngoài ra còn có gradient, trạng thái optimizer như momentum hoặc adaptive statistics, bộ nhớ tạm trong kernel và các tensor phụ trợ khác. Vì vậy, dung lượng VRAM thường trở thành rào cản đầu tiên khi mở rộng mô hình.

Bộ nhớ càng trở nên quan trọng trong các mô hình transformer sâu và rộng. Độ dài chuỗi lớn, batch lớn hoặc số đầu attention lớn đều có thể làm lượng activation tăng rất nhanh. Điều này giải thích vì sao cùng một mô hình, suy luận thường nhẹ hơn huấn luyện rất nhiều về bộ nhớ.

### 5.2 Phân tầng bộ nhớ

Phần cứng hiện đại có cấu trúc bộ nhớ phân tầng: RAM của hệ thống, VRAM của GPU, shared memory và register bên trong GPU. Mỗi mức có độ trễ và băng thông khác nhau. Một hệ thống AI hiệu quả cần đưa dữ liệu đến gần nơi tính toán nhất có thể. Nếu dữ liệu phải thường xuyên di chuyển giữa các mức xa nhau, hiệu năng sẽ giảm rõ rệt.



**Hình 6:** Phân tầng bộ nhớ và liên kết giữa các GPU trong hệ AI hiện đại.

Hình 6 cho thấy một mô hình đơn giản của phân tầng bộ nhớ và liên kết giữa các GPU. Trong thực tế, quan hệ giữa các mức bộ nhớ này quyết định cả khả năng song song hóa lẫn cách thiết kế kernel.

### 5.3 Interconnect và topology phần cứng

Không phải mọi hệ nhiều GPU đều giống nhau. Một máy có nhiều GPU nối bằng NVLink có đặc điểm khác với một cụm node nối với nhau qua Ethernet hoặc InfiniBand. Băng thông và độ trễ của interconnect ảnh hưởng trực tiếp đến hiệu quả của all-reduce, model parallelism và tensor parallelism.

Điều này có nghĩa là cùng một chiến lược song song hóa, nhưng khi đặt trên hai hạ tầng phần cứng khác nhau, hiệu quả có thể thay đổi rất mạnh. Vì vậy, mapping trong AI hiện đại phải được hiểu như một quyết định gắn chặt với topology phần cứng chứ không chỉ là gán tiến trình vào thiết bị theo nghĩa trừu tượng.

### 5.4 Các chiến lược giảm áp lực bộ nhớ

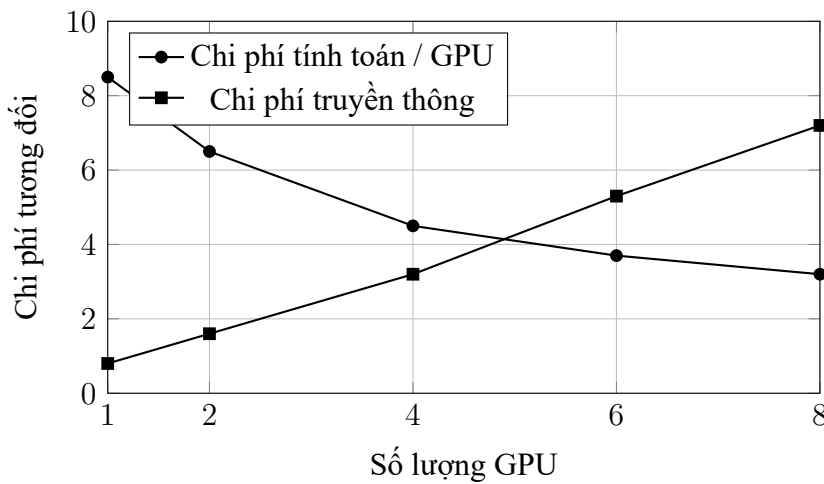
Để vượt qua ràng buộc bộ nhớ, nhiều kỹ thuật đã được phát triển như mixed precision training, activation checkpointing, optimizer sharding, CPU offloading, parameter sharding và sequence packing. Những kỹ thuật này cho thấy tối ưu bộ nhớ không còn là bài toán cục bộ bên trong một thiết bị, mà là bài toán phối hợp trên toàn hệ thống.

## 6 Các đánh đổi ở mức hệ thống

Các yếu tố đã phân tích ở trên không tồn tại độc lập, mà tạo thành một không gian đánh đổi đa chiều giữa hiệu năng hệ thống và chất lượng học.

### 6.1 Compute và communication

Khi tăng số lượng GPU, phần việc tính toán trên mỗi GPU thường giảm. Tuy nhiên, số lượng và tần suất truyền thông lại tăng. Điều này tạo ra một điểm gãy trong hiệu quả mở rộng: ban đầu, thêm GPU giúp tăng tốc đáng kể, nhưng sau một ngưỡng nhất định, lợi ích biên giảm xuống nhanh.

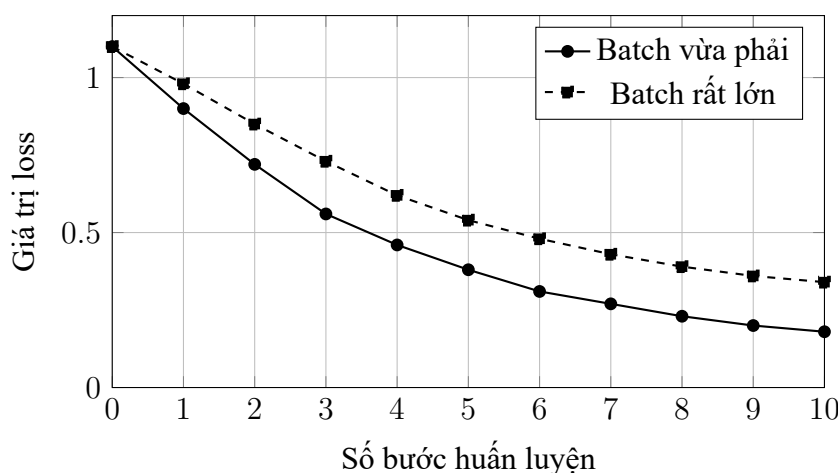


**Hình 7:** Đánh đổi giữa chi phí tính toán và chi phí truyền thông khi mở rộng số lượng GPU.

Hình 7 cho thấy xu hướng khái quát của đánh đổi này. Một chiến lược song song hóa tốt không phải là chiến lược dùng nhiều GPU nhất, mà là chiến lược đặt hệ thống tại vùng tối ưu giữa compute reduction và communication overhead.

### 6.2 Batch size và convergence

Batch size lớn thường giúp tăng throughput và giảm số lần đồng bộ trên mỗi mẫu dữ liệu. Tuy nhiên, batch rất lớn làm giảm mức độ nhiễu của gradient, có thể ảnh hưởng đến quỹ đạo tối ưu hóa và chất lượng tổng quát hóa. Trong nhiều trường hợp, batch lớn đòi hỏi điều chỉnh learning rate, warmup và lịch huấn luyện phức tạp hơn.



**Hình 8:** Minh họa đánh đổi giữa kích thước batch lớn và tốc độ/chất lượng hội tụ.

Hình 8 minh họa rằng nhanh hơn về mặt hệ thống không đồng nghĩa tốt hơn về mặt học. Đây là một trong những lý do quan trọng khiến song song hóa trong AI phải được đánh giá bằng cả chỉ số hiệu năng lẫn chỉ số học.

### 6.3 Bộ nhớ và khả năng mở rộng

Càng mở rộng mô hình, càng phải chia nhỏ nó sang nhiều thiết bị hoặc dùng nhiều kỹ thuật giảm footprint bộ nhớ. Nhưng việc đó lại kéo theo truyền thông và tăng độ phức tạp thực thi. Vì vậy, bộ nhớ và scalability luôn đi cùng nhau trong một đánh đổi liên tục.

### 6.4 Throughput và chất lượng học

Trong môi trường công nghiệp, throughput thường là mục tiêu được nhấn mạnh vì nó gắn với thời gian huấn luyện và chi phí vận hành. Tuy nhiên, một hệ thống đạt throughput cao nhưng làm mô hình hội tụ chậm hơn hoặc tổng quát hóa kém hơn chưa chắc là lựa chọn tốt. Từ góc nhìn học thuật, đây là một điểm rất đáng nhấn mạnh: đánh giá parallel computing trong AI phải là đánh giá đa chiều.

## 7 Framework và hệ thống AI phân tán hiện đại

Các framework AI hiện đại như PyTorch Distributed, DeepSpeed, Megatron-LM và Horovod đã trừu tượng hóa phần lớn chi tiết của huấn luyện phân tán, cho phép người dùng triển khai song song hóa mà không cần trực tiếp xử lý các primitive truyền thông ở mức thấp. Tuy nhiên, về bản chất, các hệ thống này vẫn dựa trên các nguyên lý cốt lõi của tính toán song song truyền thống trong HPC.

Cụ thể, Distributed Data Parallel (DDP) trong PyTorch thực hiện song song hóa theo mô hình SPMD, trong đó mỗi GPU xử lý một phần dữ liệu độc lập và đồng bộ gradient thông qua các phép toán collective như all-reduce. Cơ chế này tương đương trực tiếp với primitive `MPI_Allreduce` trong lập trình MPI.

Ở cấp độ intra-node, việc khai thác nhiều luồng trong CPU hoặc nhiều lõi trong GPU có thể được liên hệ với mô hình shared-memory trong OpenMP, trong đó các luồng cùng truy cập dữ liệu và được đồng bộ thông qua các điểm barrier.

Pipeline parallelism trong AI cũng có thể được diễn giải như một dạng task parallelism, nơi các giai đoạn xử lý được tổ chức thành pipeline và thực thi chồng lắp theo thời gian.

Việc trừu tượng hóa ở mức framework không có nghĩa là các nguyên lý bên dưới biến mất. Ngược lại, các framework hiện đại chỉ che giấu độ phức tạp triển khai, trong khi hiệu năng thực tế vẫn phụ thuộc mạnh vào cách ánh xạ phép toán lên phần cứng.

Việc tối ưu hóa GPU là một ví dụ rõ ràng. Các phép toán như `torch.matmul` hoặc các lớp convolution như `nn.Conv2d` thực chất được ánh xạ xuống các thư viện tính toán tối ưu như cuBLAS và cuDNN, trong đó các kỹ thuật như tiling đóng vai trò trung tâm. Các thư viện này tự động chia nhỏ các ma trận hoặc tensor thành các tile, nạp chúng vào shared memory và thực hiện tính toán với mức độ tái sử dụng dữ liệu cao.

Do đó, mặc dù người lập trình PyTorch chỉ cần gọi các hàm ở mức cao, hiệu năng thực tế của các phép toán này phụ thuộc mạnh vào cách các kernel bên dưới tổ chức truy cập bộ nhớ và áp dụng các chiến lược như tiling.

Trong bối cảnh huấn luyện AI, điều này có nghĩa là các quyết định ở mức mô hình, chẳng hạn như kích thước batch, kích thước tensor hoặc cách tổ chức dữ liệu, có thể ảnh hưởng trực tiếp đến mức độ hiệu quả của các kernel GPU, thông qua các cơ chế như memory coalescing và reuse trong shared memory.

**Nói cách khác, các framework như PyTorch không loại bỏ nhu cầu hiểu về tối ưu GPU, mà che giấu nó dưới các lớp trừu tượng, trong khi hiệu năng thực tế vẫn phụ thuộc vào các nguyên lý cơ bản của tính toán song song và quản lý bộ nhớ trên GPU.**

Do đó, có thể xem các framework AI hiện đại không phải là sự thay thế các mô hình lập trình song song truyền thống, mà là một lớp trừu tượng hóa ở mức cao hơn, giúp triển khai các nguyên lý này thuận tiện hơn trong bối cảnh dữ liệu và mô hình quy mô lớn. Điều này cho thấy rằng tối ưu hiệu năng trong AI không chỉ là lựa chọn mô hình, mà còn là sự tương tác giữa biểu diễn toán học và cách thực thi trên phần cứng.

## 7.1 Từ PyTorch DistributedDataParallel đến các primitive của MPI

Trong các hệ thống AI hiện đại, việc huấn luyện mô hình trên nhiều GPU hoặc nhiều nút thường được thực hiện thông qua các framework như PyTorch Distributed. Một trong những cơ chế phổ biến nhất là *DistributedDataParallel (DDP)*, cho phép song song hóa quá trình huấn luyện một cách hiệu quả mà không yêu cầu lập trình viên trực tiếp xử lý các chi tiết truyền thông.

### Mức abstraction: PyTorch DistributedDataParallel

Ví dụ đơn giản dưới đây minh họa cách sử dụng DDP trong PyTorch:

```
model = DistributedDataParallel(model)

for data in dataloader:
    output = model(data)
    loss = criterion(output)
    loss.backward()
    optimizer.step()
```

Ở mức abstraction, cơ chế DDP tự động thực hiện các bước sau:

- Phân chia dữ liệu đầu vào thành các mini-batch và phân phối cho từng GPU
- Thực hiện forward và backward propagation song song
- Tự động đồng bộ gradient giữa các GPU sau mỗi bước lan truyền ngược

Nhờ đó, lập trình viên có thể tận dụng sức mạnh của hệ thống phân tán mà không cần trực tiếp lập trình các thao tác truyền thông phức tạp.

### Bóc tách cơ chế: Đồng bộ gradient như một phép All-Reduce

Mặc dù DDP che giấu phần lớn chi tiết triển khai, cơ chế cốt lõi của nó vẫn dựa trên một phép toán quen thuộc trong HPC: *collective communication*, cụ thể là All-Reduce.

Quy trình huấn luyện có thể được diễn giải như sau:

1. Mỗi GPU nhận một phần dữ liệu và tính toán gradient cục bộ
2. Các gradient này được gửi đi để tổng hợp toàn cục
3. Kết quả tổng hợp được phân phối lại cho tất cả các GPU

#### 4. Mỗi GPU cập nhật mô hình với cùng một gradient

Cơ chế này đảm bảo rằng tất cả các bản sao của mô hình luôn duy trì trạng thái nhất quán trong quá trình huấn luyện.

#### **Liên hệ trực tiếp với MPI**

Các bước trên tương đương trực tiếp với việc sử dụng primitive `MPI_Allreduce` trong lập trình MPI:

```
MPI_Allreduce(local_grad, global_grad, MPI_SUM)
```

Trong đó:

- `local_grad`: gradient cục bộ của từng tiến trình
- `global_grad`: gradient sau khi tổng hợp toàn cục

Sau khi thực hiện `Allreduce`, mỗi tiến trình đều nhận được cùng một giá trị gradient, tương tự cơ chế đồng bộ trong DDP.

Do đó, có thể thấy rằng `DistributedDataParallel` thực chất là một lớp trừu tượng hóa ở mức cao của mô hình SPMD trong MPI, trong đó mỗi GPU tương ứng với một tiến trình độc lập.

#### **Vai trò của backend truyền thông**

Trong thực tế, PyTorch không trực tiếp gọi MPI, mà sử dụng các backend truyền thông như NCCL cho GPU hoặc Gloo cho CPU và mạng chung. Tuy nhiên, các backend này đều cung cấp các primitive tương tự MPI, đặc biệt là các phép toán collective như broadcast, reduce và all-reduce.

Điều này cho thấy rằng, mặc dù công nghệ triển khai đã thay đổi, các nguyên lý cơ bản của truyền thông song song vẫn được giữ nguyên.

#### **Góc nhìn lập trình**

Từ góc độ lập trình viên, việc sử dụng DDP giúp đơn giản hóa đáng kể quá trình phát triển hệ thống AI phân tán. Tuy nhiên, việc hiểu rõ cơ chế bên dưới, đặc biệt là các primitive như `Allreduce`, giúp người phát triển có khả năng:

- Phân tích và tối ưu hiệu năng hệ thống

- Giảm thiểu chi phí truyền thông
- Thiết kế chiến lược song song phù hợp với bài toán

Có thể thấy rằng, các framework AI hiện đại không thay thế các mô hình lập trình song song truyền thống, mà đóng vai trò là lớp trừu tượng hóa giúp đơn giản hóa việc sử dụng chúng.

Việc chuyển từ MPI sang PyTorch Distributed không phải là sự thay đổi về bản chất, mà là sự nâng cấp về mức độ trừu tượng, trong đó các nguyên lý như phân chia dữ liệu, truyền thông và đồng bộ vẫn giữ vai trò trung tâm.

| <b>PyTorch / AI abstraction</b>         | <b>MPI / NCCL primitive tương đương</b>                            |
|-----------------------------------------|--------------------------------------------------------------------|
| DistributedDataParallel (DDP)           | Mô hình SPMD (Single Program Multiple Data) trong MPI              |
| Phân chia mini-batch (data splitting)   | Data partitioning giữa các process                                 |
| Forward + Backward propagation phân tán | Thực thi song song trên nhiều tiến trình (distributed computation) |
| Gradient synchronization                | MPI_Allreduce / NCCL AllReduce                                     |
| Broadcast tham số mô hình ban đầu       | MPI_Bcast / NCCL Broadcast                                         |
| Thu thập kết quả từ nhiều worker        | MPI_Gather / MPI_Reduce                                            |
| Song song nhiều GPU trên cùng node      | Shared-memory parallelism / OpenMP                                 |
| Pipeline Parallelism                    | Task parallelism / pipeline execution                              |
| Torch.distributed backend (NCCL, Gloo)  | MPI runtime / communication layer                                  |
| Load balancing giữa các worker          | Phân phối công việc (work distribution) trong MPI/OpenMP           |

Bảng 3: Đối chiếu giữa abstraction trong PyTorch và các primitive trong MPI/NCCL.

## 8 Góc nhìn AI-PCAM

Từ các phân tích trên, có thể nhận thấy rằng các chiến lược song song hóa hiện tại đã đạt nhiều tiến bộ về mặt kỹ thuật, nhưng vẫn thiếu một khung khái niệm thống nhất liên kết giữa hiệu năng hệ thống và quá trình học của mô hình.

### 8.1 Tổng quan các hướng tiếp cận song song trong AI hiện đại

Trong những năm gần đây, song song hóa trong AI đã trở thành một chủ đề trung tâm không chỉ trong cộng đồng học máy mà còn trong hệ thống phân tán và tính toán hiệu năng cao. Nhiều hướng tiếp cận đã được đề xuất nhằm giải quyết bài toán mở rộng mô hình và dữ liệu, đặc biệt trong bối cảnh các mô hình deep learning ngày càng gia tăng về quy mô.

Một hướng nghiên cứu quan trọng tập trung vào tối ưu data parallelism thông qua các biến thể của thuật toán đồng bộ gradient. Các công trình này nhằm giảm chi phí truyền thông bằng cách nén gradient, truyền gradient thưa hoặc sử dụng các kỹ thuật như local SGD, trong đó gradient được tích lũy cục bộ trước khi đồng bộ. Những phương pháp này cho thấy có thể giảm tần suất truyền thông mà vẫn duy trì chất lượng học ở mức chấp nhận được.

Song song với đó, model parallelism và tensor parallelism được nghiên cứu mạnh trong bối cảnh các mô hình transformer quy mô lớn. Các hệ thống như Megatron-LM [1] cho thấy việc chia các phép toán ma trận lớn theo chiều tensor cho phép huấn luyện mô hình với hàng trăm tỷ tham số. Tuy nhiên, các hệ thống này cũng nhấn mạnh rằng chi phí truyền thông nội bộ giữa các GPU là yếu tố quyết định hiệu năng thực tế.

Pipeline parallelism cũng được phát triển với nhiều biến thể như GPipe [2], PipeDream và các kỹ thuật scheduling phức tạp nhằm giảm pipeline bubble. Các nghiên cứu này chỉ ra rằng việc tổ chức forward và backward pass có ảnh hưởng đáng kể đến hiệu suất tổng thể, đặc biệt khi số lượng stage lớn.

Một hướng khác tập trung vào tối ưu bộ nhớ. Các kỹ thuật như ZeRO (Zero Redundancy Optimizer) [3] chia nhỏ trạng thái optimizer, gradient và tham số trên nhiều GPU nhằm giảm footprint bộ nhớ, qua đó cho phép huấn luyện các mô hình lớn hơn mà không cần tăng dung lượng VRAM trên từng thiết bị.

Ở cấp độ hệ thống, các framework như DeepSpeed, FairScale và PyTorch Distributed cung cấp các primitive để kết hợp nhiều hình thức song song hóa. Tuy nhiên, phần lớn các hệ thống này vẫn yêu cầu cấu hình thủ công hoặc bán tự động, chưa đạt đến mức tối ưu hóa toàn diện một cách tự động.

Nhìn chung, các nghiên cứu hiện tại đã đạt nhiều tiến bộ về mặt kỹ thuật, nhưng vẫn thiếu một khung lý thuyết thống nhất để kết nối các quyết định song song hóa với động học học tập của mô hình.

## 8.2 Khoảng trống nghiên cứu

Mặc dù lĩnh vực huấn luyện phân tán và AI quy mô lớn đã đạt nhiều tiến bộ, vẫn tồn tại một số khoảng trống đáng chú ý.

Thứ nhất, nhiều hệ thống hiện nay ưu tiên tối ưu hiệu năng nhưng chưa tích hợp chặt với động học học tập. Các quyết định như chia batch, nén gradient hay chu kỳ đồng bộ thường được đưa ra theo logic hệ thống trước, sau đó mới kiểm tra tác động đến chất lượng học.

Thứ hai, nhiều chiến lược còn mang tính tĩnh. Một cấu hình tối ưu ở giai đoạn đầu huấn luyện chưa chắc phù hợp ở giai đoạn sau. Trong thực tế, cần các chiến lược song song hóa thích nghi theo thời gian, theo đặc trưng dữ liệu hoặc trạng thái bộ nhớ.

Thứ ba, vẫn thiếu một khung khái niệm nối liền tư duy song song hóa cổ điển với AI hiện đại. Trong HPC, các nguyên lý Partitioning, Communication, Agglomeration và Mapping đóng vai trò nền tảng. Trong AI, các quyết định tương tự vẫn tồn tại, nhưng cần được diễn giải lại dưới các ràng buộc mới như hội tụ, dữ liệu không đồng nhất và batch dynamics.

**Do đó, vấn đề đặt ra không chỉ là tối ưu hệ thống, mà là tìm một cách nhìn thống nhất để kết nối giữa HPC truyền thống và các hệ thống AI hiện đại.**

## 8.3 Nhắc lại mô hình PCAM

PCAM là một khuôn khổ thiết kế song song cổ điển trong HPC [4], gồm bốn bước: *Partitioning*, *Communication*, *Agglomeration* và *Mapping*. Mô hình này giúp phân rã bài toán, tổ chức truyền thông, gom nhóm tác vụ và ánh xạ lên tài nguyên tính toán.

Trong bối cảnh AI hiện đại, các nguyên lý này vẫn còn giá trị, nhưng không còn đủ để mô tả đầy đủ hệ thống. Các quyết định song song hóa giờ đây không chỉ phụ thuộc vào cấu trúc tính toán, mà còn gắn chặt với quá trình học của mô hình, bao gồm các yếu tố như hội tụ, độ ổn định và chất lượng tối ưu hóa.

## 8.4 Từ PCAM cổ điển đến AI-PCAM

Trong phạm vi báo cáo này, AI-PCAM được sử dụng như một *khung phân tích* (analytical lens) nhằm diễn giải các cơ chế song song trong AI hiện đại dưới góc nhìn mở rộng của PCAM truyền thống.

Có thể xem AI-PCAM như một cách diễn giải chuyển trọng tâm từ “parallelizing computation” sang “co-design between learning and parallel execution”.

**Insight chính:** Các quyết định song song hóa trong AI không thể được đánh giá độc lập theo tiêu chí hiệu năng, mà phải được xem xét trong mối quan hệ trực tiếp với động học học tập của

mô hình.

Trong cách nhìn này:

- Partitioning không chỉ là chia tác vụ, mà bao gồm chia dữ liệu, mô hình, tensor và trạng thái optimizer
- Communication không chỉ là truyền dữ liệu, mà là đồng bộ gradient, activation và tín hiệu huấn luyện
- Agglomeration không chỉ nhằm giảm overhead, mà còn bao gồm các quyết định như gradient accumulation, micro-batching và fusion
- Mapping không chỉ là ánh xạ tiến trình, mà là ánh xạ lên topology phần cứng nhiều lớp

Ví dụ trong multi-head attention, việc phân chia head có thể được xem như một dạng partitioning theo tensor, trong đó các quyết định phân chia không chỉ dựa trên kích thước, mà còn phụ thuộc vào vai trò của từng head trong quá trình học.

Theo định luật Amdahl, tốc độ tăng tốc của hệ song song bị giới hạn bởi phần tuần tự. Trong AI, điều này thể hiện rõ ở dependency chain giữa các layer trong quá trình backpropagation.

## 8.5 Không gian quyết định trong AI-PCAM

Để cụ thể hóa cách diễn giải AI-PCAM, có thể xem các quyết định song song hóa như một không gian thiết kế đa chiều.

- Chọn chiến lược partition: data, model, tensor hoặc pipeline
- Quyết định tần suất và cơ chế communication
- Điều chỉnh batch size và micro-batching
- Lựa chọn cách ánh xạ lên phần cứng

Mỗi quyết định không chỉ ảnh hưởng đến hiệu năng, mà còn tác động trực tiếp đến hội tụ và chất lượng học.

**Nói cách khác, không gian thiết kế trong AI-PCAM là một bài toán cân bằng giữa hiệu năng hệ thống và hành vi học của mô hình.**

## 8.6 Minh họa trực quan AI-PCAM

Kích thước batch là một trong những yếu tố quan trọng ảnh hưởng trực tiếp đến hiệu năng khai thác GPU trong huấn luyện AI. Khi batch size nhỏ, số lượng dữ liệu được xử lý trong mỗi lần gọi kernel không đủ lớn để tận dụng hết số lượng thread và warp trên GPU, dẫn đến mức độ song song thấp và tài nguyên tính toán bị lãng phí. Ngoài ra, chi phí khởi tạo kernel và truyền dữ liệu có thể chiếm tỷ trọng lớn hơn trong tổng thời gian thực thi.

Ngược lại, khi batch size tăng, các phép toán tensor như matrix multiplication và convolution có thể được tổ chức thành các kernel lớn hơn, tăng mức độ song song và cải thiện hiệu quả sử dụng GPU. Điều này cũng giúp tăng khả năng tái sử dụng dữ liệu trong shared memory, từ đó giảm chi phí truy cập global memory.

Tuy nhiên, batch size lớn cũng kéo theo yêu cầu bộ nhớ cao hơn và có thể ảnh hưởng đến động học hội tụ của mô hình. Do đó, việc lựa chọn batch size là một bài toán cân bằng giữa hiệu năng hệ thống và chất lượng học. Trong thực tế, batch size không chỉ là một tham số học máy, mà còn là một tham số hệ thống, quyết định mức độ khai thác song song của GPU.

Từ góc nhìn AI-PCAM, các ví dụ thực nghiệm này cho thấy rằng:

- Các quyết định về batch size ảnh hưởng đến cả throughput và động học hội tụ
- Việc tổ chức tensor ảnh hưởng đến hiệu năng hệ thống và độ ổn định huấn luyện
- Mapping lên GPU không chỉ là vấn đề kỹ thuật, mà ảnh hưởng trực tiếp đến hiệu quả học

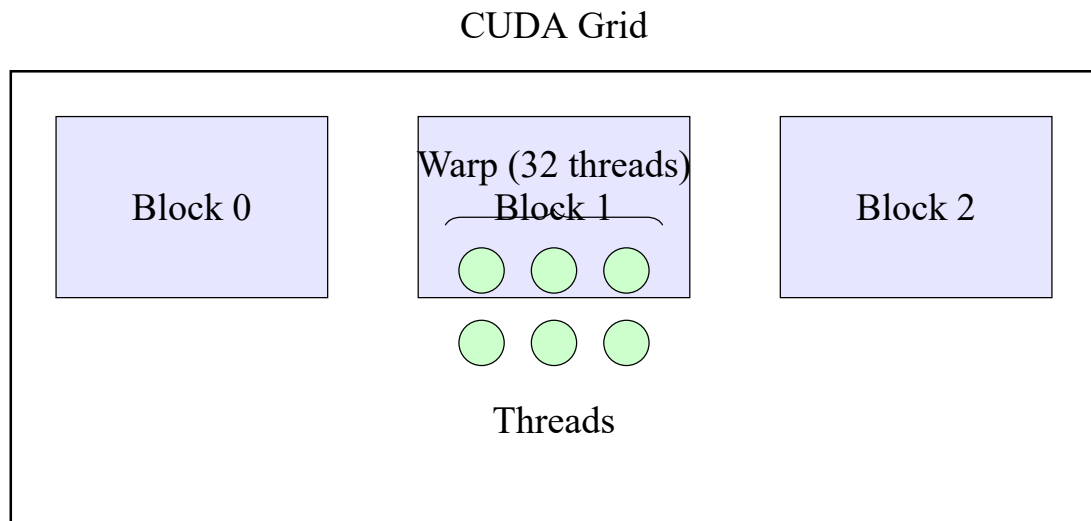
**Do đó, ngay cả trong thiết lập một GPU, song song hóa trong AI vẫn là một bài toán kết hợp giữa hệ thống và học máy, phù hợp với cách diễn giải của AI-PCAM.**

Các thí nghiệm này cũng cho thấy rằng, GPU không chỉ là thiết bị tăng tốc, mà là nền tảng thực thi song song cốt lõi của AI hiện đại.

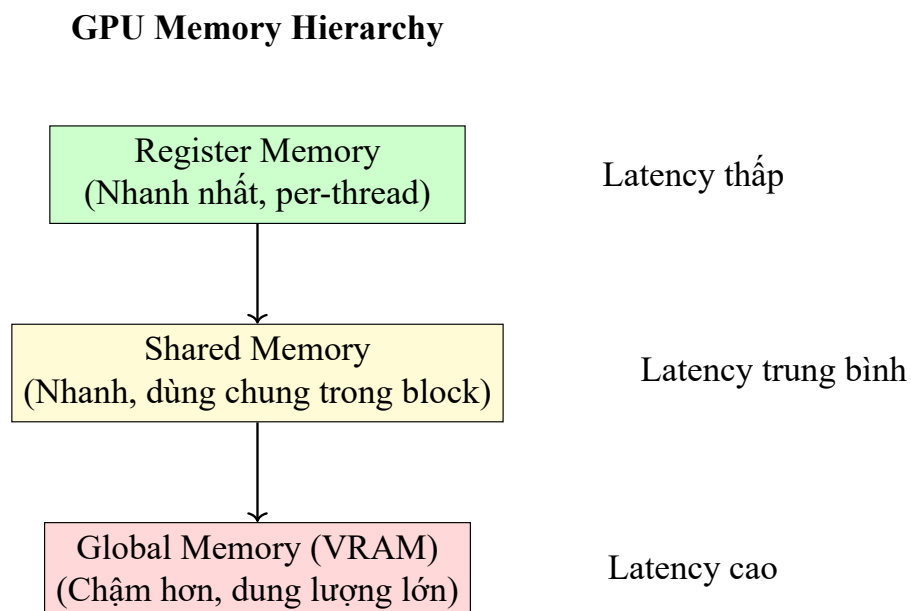
Hình 9 minh họa mô hình thực thi song song trên GPU theo CUDA. Một chương trình được triển khai dưới dạng một grid gồm nhiều block, mỗi block bao gồm nhiều thread thực thi song song. Các thread được nhóm thành các warp, thường gồm 32 thread, là đơn vị thực thi cơ bản trên GPU. Các warp này được lập lịch và thực thi đồng thời trên các nhân CUDA, giúp khai thác mức song song rất cao trong các phép toán tensor.

Trong các framework như PyTorch, các phép toán tensor được ánh xạ xuống các kernel CUDA, trong đó mỗi kernel được thực thi bởi hàng nghìn thread theo cấu trúc grid–block–thread như trên.

Hình 10 minh họa phân tầng bộ nhớ trong GPU. Register memory là bộ nhớ nhanh nhất, được gắn trực tiếp với từng thread, cho phép truy cập với độ trễ rất thấp. Shared memory được chia



**Hình 9:** Mô hình song song GPU theo CUDA: Grid gồm nhiều Block, mỗi Block gồm nhiều Thread; các thread được nhóm thành warp để thực thi song song trên GPU.

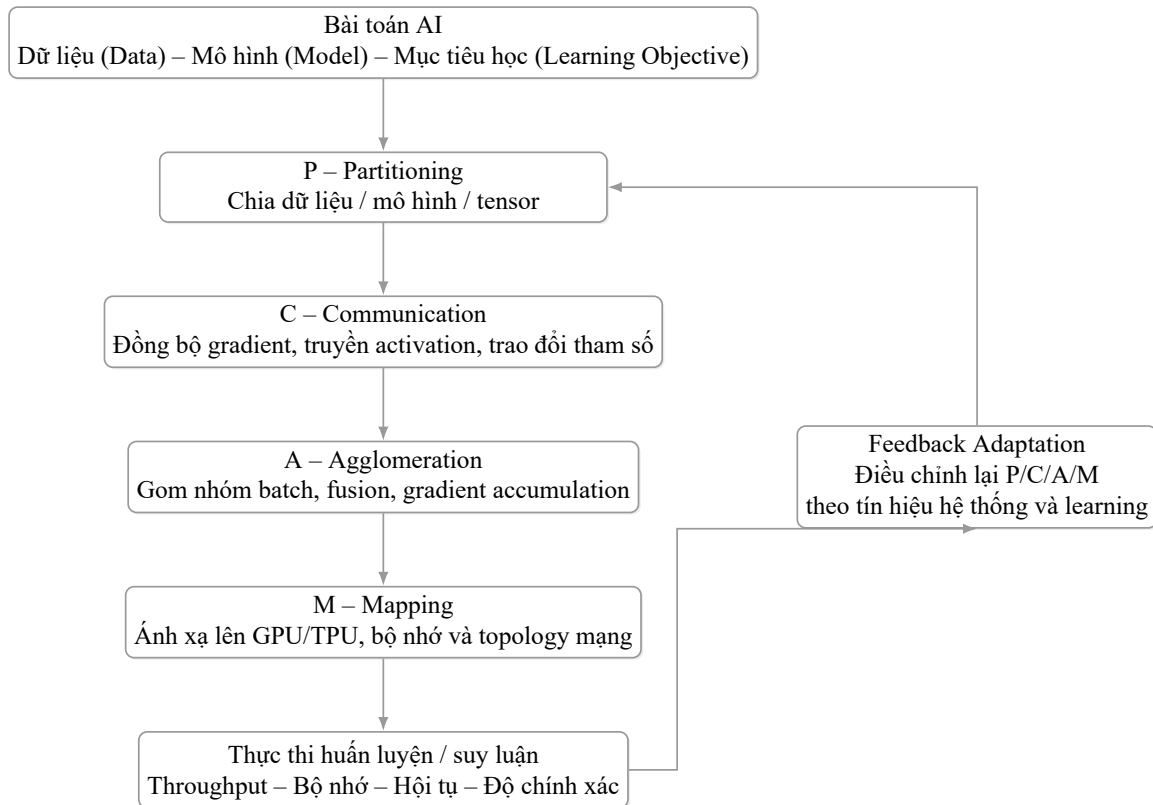


**Hình 10:** Phân tầng bộ nhớ GPU: register (nhanh nhất), shared memory và global memory (VRAM), với sự đánh đổi giữa tốc độ truy cập và dung lượng lưu trữ.

sẽ giữa các thread trong cùng một block, cho phép trao đổi dữ liệu nhanh hơn so với global memory.

Global memory (VRAM) có dung lượng lớn nhưng độ trễ cao hơn đáng kể. Do đó, hiệu năng của các kernel CUDA phụ thuộc mạnh vào việc tối ưu truy cập bộ nhớ, giảm số lần truy cập global memory và tận dụng shared memory khi có thể.

Trong các framework như PyTorch, việc tổ chức tensor và layout bộ nhớ có ảnh hưởng trực tiếp đến hiệu năng tính toán trên GPU.



*AI-PCAM mở rộng PCAM cổ điển bằng cách gắn thiết kế song song với động học học tập và phản hồi thích nghi.*

**Hình 11:** Khung AI-PCAM cho hệ AI hiện đại.

Hình 11 minh họa khung AI-PCAM ở mức khái niệm, trong đó các thành phần PCAM được mở rộng và liên kết với vòng phản hồi từ quá trình học.

## 8.7 Nhận xét tổng hợp

Về bản chất, cách diễn giải này cho thấy rằng tính toán song song trong AI không chỉ là bài toán tối ưu hiệu năng, mà là một quá trình đồng thiết kế giữa thuật toán học và hệ thống thực thi.

**Tóm lại, AI-PCAM trong báo cáo này không phải là một framework chính thức, mà là một cách khái niệm hóa nhằm giúp hiểu rõ hơn mối liên hệ giữa HPC truyền thống và AI hiện**

**đại, đặc biệt trong bối cảnh các hệ thống AI ngày càng mở rộng về quy mô và độ phức tạp.**

## 9 Góc nhìn lập trình: Liên hệ MPI và OpenMP

Mặc dù các framework AI hiện đại như PyTorch Distributed, DeepSpeed hay Megatron-LM đã trừu tượng hóa phần lớn chi tiết triển khai, các nguyên lý cốt lõi của chúng vẫn dựa trên các mô hình lập trình song song truyền thống đã được nghiên cứu sâu trong HPC, đặc biệt là MPI và OpenMP.

### 9.1 Data Parallelism và mô hình SPMD trong MPI

Trong data parallelism, mô hình học được sao chép trên nhiều thiết bị tính toán, trong khi dữ liệu được phân chia thành các phần nhỏ và xử lý độc lập. Cách tiếp cận này tương đồng trực tiếp với mô hình SPMD trong MPI, nơi mỗi tiến trình thực thi cùng một chương trình trên các tập dữ liệu khác nhau.

Cơ chế đồng bộ gradient trong huấn luyện phân tán có thể được xem là một dạng collective communication, tương đương với các primitive như `MPI_Allreduce`. Mỗi tiến trình tính toán gradient cục bộ và sau đó thực hiện phép reduce rồi phân phối lại kết quả để đảm bảo tính nhất quán của mô hình trên toàn hệ thống.

Do đó, có thể nhận thấy rằng các thư viện như NCCL hay Horovod thực chất là các lớp tối ưu hóa chuyên biệt cho các phép toán collective trong bối cảnh AI, nhưng về bản chất vẫn kế thừa trực tiếp từ các khái niệm trong MPI [5].

### 9.2 Song song trong bộ nhớ chia sẻ và OpenMP

Ở cấp độ intra-node, khi nhiều luồng tính toán cùng truy cập một không gian bộ nhớ chung, mô hình này có thể được liên hệ với lập trình OpenMP. Trong OpenMP, các luồng được sinh ra để thực hiện song song các vòng lặp hoặc các khối lệnh, đồng thời chia sẻ bộ nhớ chung.

Tương tự, trong các hệ thống AI hiện đại, việc khai thác song song trong một GPU hoặc trong nhiều lõi CPU thường dựa trên cơ chế chia sẻ bộ nhớ, trong đó các luồng thực thi đồng thời trên các phần dữ liệu khác nhau. Các khái niệm như *barrier*, *synchronization* hay *critical section* trong OpenMP cũng có thể được quan sát dưới dạng các điểm đồng bộ trong quá trình huấn luyện mô hình.

### 9.3 Pipeline Parallelism và song song theo tác vụ

Pipeline parallelism có thể được diễn giải dưới góc nhìn của song song theo tác vụ (task parallelism), trong đó các giai đoạn xử lý khác nhau được phân bổ cho các đơn vị tính toán riêng biệt. Mỗi đơn vị xử lý một phần của luồng công việc và chuyển tiếp kết quả cho giai đoạn tiếp theo.

Mô hình này tương đồng với các pipeline trong lập trình song song truyền thống, nơi các tác vụ được chồng lấp theo thời gian để tối đa hóa mức sử dụng tài nguyên. Tuy nhiên, trong bối cảnh AI, sự tồn tại của các phụ thuộc dữ liệu giữa các layer làm phát sinh hiện tượng *pipeline bubble*, phản ánh rõ giới hạn của việc song song hóa theo chiều sâu.

### 9.4 Từ MPI/OpenMP đến các framework AI hiện đại

Các framework AI hiện đại có thể được xem là một lớp trừu tượng hóa ở mức cao hơn của các mô hình lập trình song song cổ điển. Thay vì yêu cầu lập trình viên trực tiếp quản lý tiến trình, truyền thông và đồng bộ, các framework này tự động hóa các thao tác như phân chia dữ liệu, đồng bộ gradient và ánh xạ tác vụ lên phần cứng.

Tuy nhiên, bản chất của các cơ chế này vẫn dựa trên các nguyên lý của MPI và OpenMP. Việc hiểu rõ các primitive như broadcast, reduce, barrier hay scatter giúp lập trình viên có cái nhìn sâu sắc hơn về cách hệ thống hoạt động, từ đó có khả năng tối ưu hóa hiệu năng trong các kịch bản huấn luyện quy mô lớn.

### 9.5 Góc nhìn của lập trình viên

Từ góc độ thực tiễn, việc nắm vững MPI và OpenMP không chỉ có ý nghĩa lý thuyết, mà còn đóng vai trò quan trọng trong việc hiểu và tối ưu các hệ thống AI hiện đại.

Các framework như PyTorch Distributed hay DeepSpeed giúp giảm đáng kể độ phức tạp trong triển khai, nhưng không loại bỏ hoàn toàn các vấn đề liên quan đến communication overhead, synchronization cost hay load balancing.

Do đó, việc tiếp cận các hệ thống AI từ góc nhìn lập trình song song giúp người phát triển không chỉ sử dụng công cụ, mà còn hiểu được cơ chế vận hành bên trong, từ đó đưa ra các quyết định thiết kế hợp lý nhằm đạt hiệu năng tối ưu.

Có thể thấy rằng, các kỹ thuật song song trong AI hiện đại không tách rời khỏi nền tảng lý thuyết của lập trình song song truyền thống. Ngược lại, chúng kế thừa và mở rộng các nguyên lý của MPI và OpenMP trong một bối cảnh mới với quy mô dữ liệu và mô hình lớn hơn nhiều.

Do đó, việc học và hiểu các mô hình lập trình song song cơ bản vẫn là nền tảng quan trọng để phát triển các hệ thống AI hiệu năng cao trong thực tế.

## 10 Case Study: Song song hóa trong Transformer và LLM

### 10.1 Cấu trúc tính toán của Transformer

Transformer là kiến trúc chủ đạo trong nhiều hệ AI hiện đại, đặc biệt là các mô hình ngôn ngữ lớn (LLM). Mỗi khối transformer bao gồm hai thành phần chính: multi-head self-attention và feed-forward network. Cả hai thành phần này đều dựa trên các phép nhân ma trận quy mô lớn, khiến chúng trở thành ứng viên lý tưởng cho song song hóa.

Trong self-attention, ba tensor  $Q$ ,  $K$ ,  $V$  được tạo ra từ đầu vào thông qua các phép biến đổi tuyến tính. Sau đó, attention score được tính bằng phép nhân  $QK^T$ , tiếp theo là softmax và nhân với  $V$ . Mỗi bước này đều có thể được phân chia theo nhiều chiều khác nhau.

### 10.2 Áp dụng data parallelism trong LLM

Trong huấn luyện LLM, data parallelism thường được dùng để mở rộng theo chiều dữ liệu. Mỗi GPU xử lý một phần batch và tính gradient cục bộ. Sau đó, gradient được đồng bộ thông qua all-reduce.

Tuy nhiên, do kích thước mô hình lớn, batch size tổng thường phải rất lớn để tận dụng nhiều GPU. Điều này kéo theo các vấn đề về convergence, đòi hỏi các kỹ thuật như learning rate scaling, warmup và adaptive optimization.

### 10.3 Áp dụng tensor parallelism

Tensor parallelism được áp dụng mạnh trong transformer bằng cách chia các ma trận lớn theo chiều cột hoặc chiều hàng. Ví dụ, ma trận trọng số trong feed-forward layer có thể được chia thành nhiều phần, mỗi phần nằm trên một GPU.

Điều này giúp giảm yêu cầu bộ nhớ trên mỗi GPU và tăng khả năng song song hóa, nhưng đòi hỏi truyền thông để ghép kết quả lại. Các phép toán như all-gather hoặc reduce-scatter thường được sử dụng trong quá trình này.

### 10.4 Kết hợp pipeline parallelism

Khi số tầng transformer lớn, pipeline parallelism được sử dụng để chia mô hình thành nhiều stage. Mỗi stage xử lý một nhóm layer và các micro-batch được đưa vào pipeline theo thời gian.

Việc kết hợp pipeline với tensor parallelism tạo ra một cấu trúc song song đa chiều, trong đó mỗi tầng vừa được chia theo chiều tensor, vừa được đặt trong pipeline.

## 10.5 Hybrid parallelism trong thực tế

Trong thực tế, các hệ thống huấn luyện LLM sử dụng hybrid parallelism, kết hợp:

- Data parallelism để chia dữ liệu
- Tensor parallelism để chia phép toán lớn
- Pipeline parallelism để chia mô hình theo stage

Cấu trúc này cho phép mở rộng mô hình lên hàng trăm hoặc hàng nghìn GPU. Tuy nhiên, việc cấu hình và tối ưu hệ thống trở nên rất phức tạp, đòi hỏi hiểu biết sâu về cả mô hình lẫn hạ tầng.

## 11 Thảo luận và hướng nghiên cứu

Từ các phân tích trên, có thể thấy rằng tính toán song song trong AI hiện đại là một chủ đề mang tính liên ngành, trong đó thiết kế hệ thống không thể tách rời khỏi lý thuyết học máy. Các quyết định song song hóa không chỉ ảnh hưởng đến hiệu năng, mà còn tác động trực tiếp đến hội tụ, độ ổn định và chất lượng tối ưu hóa của mô hình.

Một hướng nghiên cứu quan trọng là phát triển các cơ chế *adaptive parallelism*, trong đó chiến lược song song hóa có thể thay đổi theo trạng thái học, đặc trưng dữ liệu hoặc điều kiện hệ thống. Điều này mở ra khả năng xây dựng các hệ thống AI có khả năng tự điều chỉnh cấu hình, thay vì phụ thuộc hoàn toàn vào thiết kế tĩnh.

Dưới góc nhìn này, AI-PCAM có thể được xem như một khung phân tích nhằm hỗ trợ việc hiểu và tổ chức không gian quyết định song song hóa. Khung này gợi ý rằng các quyết định về partitioning, communication, agglomeration và mapping cần được xem xét đồng thời trong mối quan hệ với quá trình học của mô hình, thay vì tách rời như trong các hệ thống truyền thống.

Ngoài các mô hình ngôn ngữ lớn, tư duy này còn có thể áp dụng cho nhiều lĩnh vực khác như hệ gợi ý, phát hiện gian lận, chấm điểm tín dụng, dự báo chuỗi thời gian hoặc phân tích dữ liệu viễn thông. Trong các bài toán này, yêu cầu không chỉ là độ chính xác, mà còn bao gồm chi phí vận hành, thời gian huấn luyện và khả năng triển khai trên hạ tầng thực tế.

Từ góc độ nghiên cứu, có thể mở rộng công trình này theo các hướng:

- Kết hợp AI-PCAM với các phương pháp học tăng cường để tự động tối ưu cấu hình song song hóa
- Xây dựng các mô hình toán học đa mục tiêu kết hợp giữa hiệu năng hệ thống và chất lượng học

- Triển khai các hệ thống prototype trên PyTorch Distributed hoặc các nền tảng tương tự
- Đánh giá thực nghiệm trên các cụm GPU quy mô lớn

Trong tương lai, các hệ thống AI có thể không chỉ được thiết kế bởi con người, mà còn có khả năng tự tối ưu cấu trúc song song của chính mình dựa trên dữ liệu và môi trường vận hành, mở ra một hướng tiếp cận mới cho thiết kế hệ thống thông minh.

## 11.1 LLM scaling laws và các giới hạn truyền thống

Việc mở rộng huấn luyện từ một GPU sang nhiều GPU không dẫn đến tăng tốc tuyến tính, mặc dù tổng năng lực tính toán danh nghĩa tăng theo số lượng thiết bị.

Nguyên nhân chính là sự xuất hiện của chi phí truyền thông và đồng bộ. Trong data parallelism, sau mỗi bước lan truyền ngược, các GPU phải trao đổi và tổng hợp gradient thông qua các phép toán như all-reduce. Chi phí này tăng theo số lượng tham số và số lượng thiết bị, và có thể trở thành nút thắt cổ chai khi mở rộng hệ thống.

Ngoài ra, việc phân chia công việc không phải lúc nào cũng cân bằng hoàn hảo. Hiện tượng *straggler* xảy ra khi một số GPU hoàn thành chậm hơn, khiến toàn bộ hệ thống phải chờ tại điểm đồng bộ. Bên cạnh đó, độ trễ khởi tạo kernel, chi phí truyền dữ liệu giữa CPU và GPU, và giới hạn băng thông của interconnect như PCIe, NVLink hoặc mạng liên nút cũng góp phần làm giảm hiệu quả mở rộng.

Từ góc độ hệ thống, khi số lượng GPU tăng, phần tính toán trên mỗi thiết bị giảm, trong khi chi phí truyền thông và đồng bộ tăng, dẫn đến một điểm cân bằng tối ưu thay vì tăng tốc vô hạn.

Điều này phản ánh một dạng mở rộng của định luật Amdahl trong bối cảnh AI hiện đại, trong đó phần tuần tự được thay thế bởi chi phí truyền thông.

Ở chiều ngược lại, các LLM scaling laws cho thấy rằng việc mở rộng kích thước mô hình, dữ liệu và compute có thể dẫn đến cải thiện có hệ thống về chất lượng mô hình. Do đó, AI hiện đại đồng thời chịu tác động của hai loại giới hạn: một bên là động lực mở rộng quy mô từ học máy, bên kia là giới hạn thực thi từ hệ thống tính toán.

Nói cách khác, LLM scaling laws trả lời câu hỏi “nên mở rộng quy mô mô hình đến đâu”, trong khi các định luật trong HPC và các giới hạn truyền thống của song song hóa trả lời câu hỏi “có thể mở rộng hiệu quả đến đâu”.

Sự tương tác giữa hai chiều này tạo thành một không gian thiết kế quan trọng trong AI hiện đại, trong đó tối ưu hóa không chỉ nằm ở mô hình, mà còn ở cách tổ chức tính toán trên hạ tầng phân tán.

## 12 Kết luận

Báo cáo đã trình bày một cái nhìn tổng thể về tính toán song song trong AI hiện đại, từ bản chất tensor của phép tính trong học sâu, các hình thức song song hóa chủ đạo, đến các ràng buộc về truyền thông, bộ nhớ, topology phần cứng và các đánh đổi ở mức hệ thống.

Thông qua việc phân tích data parallelism, model parallelism, pipeline parallelism, tensor parallelism và huấn luyện phân tán, có thể thấy rằng hiệu năng của hệ thống AI không chỉ phụ thuộc vào năng lực tính toán, mà còn phụ thuộc mạnh vào cách tổ chức và điều phối tính toán.

Một điểm quan trọng được rút ra là các quyết định trong AI hiện đại không thể tách rời giữa thuật toán và hệ thống. Các lựa chọn như batch size, kiến trúc mô hình hay chiến lược song song hóa đều có tác động đồng thời đến hiệu năng tính toán và động học học tập của mô hình. Điều này cho thấy rằng AI hiện đại không chỉ là bài toán mô hình, mà là bài toán đồng thiết kế giữa học máy và hệ thống tính toán.

Từ góc nhìn này, các scaling laws trong học máy và các định luật song song trong HPC có thể được xem như hai mặt của cùng một vấn đề: một bên thúc đẩy việc mở rộng quy mô mô hình, trong khi bên còn lại đặt ra các giới hạn về khả năng thực thi trên hệ thống.

Trong báo cáo này, cách tiếp cận AI-PCAM được sử dụng như một khung phân tích nhằm kết nối hai góc nhìn đó, cho phép hiểu rõ hơn mối quan hệ giữa thiết kế song song và quá trình học.

Trong tương lai, việc phát triển các hệ thống AI hiệu quả sẽ phụ thuộc ngày càng nhiều vào khả năng kết hợp chặt chẽ giữa thuật toán học và kiến trúc hệ thống. **AI hiện đại không chỉ là vấn đề tính toán nhiều hơn, mà là vấn đề tính toán đúng cách trên hệ thống phù hợp.**

# Phụ lục A

## Nguồn tài liệu và hình thành báo cáo

Phụ lục này trình bày các tài liệu chính được sử dụng trong báo cáo và cách chúng góp phần hình thành cách tiếp cận được trình bày, đặc biệt là góc nhìn AI-PCAM.

### A.0.1 Câu hỏi khởi đầu

Quá trình nghiên cứu bắt đầu từ câu hỏi: *Làm thế nào để triển khai song song hóa hiệu quả trong huấn luyện mô hình AI trên GPU, đặc biệt trong bối cảnh PyTorch và các framework hiện đại?*

Từ câu hỏi này, các vấn đề về data parallelism, đồng bộ gradient và cơ chế thực thi trên GPU được xem xét như những điểm khởi đầu quan trọng.

### A.0.2 Tài liệu nền tảng về song song hóa

Tài liệu có ảnh hưởng nền tảng là:

- Foster, I. (1995), *Designing and Building Parallel Programs*

Cuốn sách này giới thiệu mô hình PCAM (Partitioning, Communication, Agglomeration, Mapping), được sử dụng như một khung tư duy cơ bản để phân tích các bài toán song song.

### A.0.3 Các tài liệu về huấn luyện phân tán trong AI

Các tài liệu tiếp theo tập trung vào huấn luyện phân tán và hệ thống AI hiện đại, bao gồm:

- PyTorch Distributed và Data Parallel
- Các nghiên cứu về Large Language Models (LLM)

- Tensor Parallelism và Pipeline Parallelism
- Các hệ thống như DeepSpeed và Megatron-LM

Những tài liệu này giúp làm rõ cách các nguyên lý song song được triển khai trong thực tế AI hiện đại.

#### A.0.4 Hình thành góc nhìn AI-PCAM

Thông qua việc kết hợp giữa mô hình PCAM cổ điển và các cơ chế huấn luyện phân tán trong AI, một cách nhìn tổng hợp được hình thành, trong đó các quyết định song song hóa được xem xét đồng thời với động học học tập của mô hình.

Trong báo cáo này, cách tiếp cận này được gọi là AI-PCAM, như một khung phân tích nhằm diễn giải mối quan hệ giữa thiết kế song song và quá trình học.

**AI-PCAM không được xem như một framework chính thức**, mà là một cách khái niệm hóa để hỗ trợ việc hiểu và tổ chức các quyết định hệ thống trong AI hiện đại.

# Phụ lục B

## Minh họa lập trình song song

Phụ lục này trình bày một số ví dụ lập trình minh họa cho các nguyên lý tính toán song song đã được đề cập trong báo cáo. Mục tiêu không phải là xây dựng một hệ thống hoàn chỉnh, mà là làm rõ bản chất của phân chia dữ liệu, truyền thông và đồng bộ.

### B.0.1 OpenMP: Song song hóa vòng lặp

**Mục tiêu:** Minh họa mô hình song song trong bộ nhớ chia sẻ (shared memory).

```
#include <stdio.h>
#include <omp.h>

int main() {
    int i;
    double sum = 0.0;
    int N = 1000000;

    #pragma omp parallel for reduction(+:sum)
    for (i = 1; i <= N; i++) {
        sum += 1.0 / i;
    }

    printf("Tong = %f\n", sum);
    return 0;
}
```

**Giải thích:**

Đoạn mã trên sử dụng OpenMP để chia vòng lặp thành nhiều luồng xử lý song song. Directive `parallel for` phân phối các iteration cho các luồng, trong khi `reduction(+:sum)` bảo đảm việc cộng dồn kết quả không gây ra race condition.

Ví dụ này thể hiện rõ đặc trưng của mô hình shared-memory parallelism: các luồng cùng làm việc trên một không gian bộ nhớ chung, nhưng cần cơ chế đồng bộ phù hợp để đảm bảo tính đúng đắn.

## B.0.2 MPI: Tính tổng phân tán

**Mục tiêu:** Minh họa mô hình message passing trong tính toán phân tán.

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char* argv[]) {
    int rank, size, i;
    int local_sum = 0, global_sum = 0;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    for (i = rank * 10 + 1; i <= (rank + 1) * 10; i++) {
        local_sum += i;
    }

    MPI_Reduce(&local_sum, &global_sum, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        printf("Tong = %d\n", global_sum);
    }

    MPI_Finalize();
    return 0;
}
```

### Giải thích:

Trong ví dụ này, mỗi tiến trình MPI xử lý một phần dữ liệu riêng, sau đó sử dụng `MPI_Reduce`

để tổng hợp kết quả về tiến trình gốc. Khác với OpenMP, các tiến trình MPI không chia sẻ bộ nhớ trực tiếp, mà trao đổi dữ liệu thông qua truyền thông điệp.

Ví dụ này minh họa rõ cách bài toán được phân rã và cách kết quả cục bộ được tổng hợp trong môi trường phân tán.

### B.0.3 MPI Allreduce: Đồng bộ gradient

**Mục tiêu:** Liên hệ trực tiếp với huấn luyện phân tán trong Deep Learning.

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char* argv[]) {
    int rank, size;
    float local_grad = rank + 1.0f;
    float global_grad;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    MPI_Allreduce(&local_grad, &global_grad, 1, MPI_FLOAT, MPI_SUM, MPI_COMM_WORLD);
    global_grad /= size;

    printf("Process %d: grad = %.2f\n", rank, global_grad);

    MPI_Finalize();
    return 0;
}
```

#### Giải thích:

Đây là một mô phỏng đơn giản của cơ chế đồng bộ gradient trong huấn luyện phân tán theo kiểu data parallelism. Mỗi tiến trình tính một gradient cục bộ, sau đó MPI\_Allreduce được sử dụng để tổng hợp và phân phối lại gradient trung bình cho tất cả các tiến trình.

Cơ chế này tương đương trực tiếp với bước đồng bộ gradient trong nhiều framework AI hiện đại, và là một cầu nối rõ ràng giữa MPI truyền thống và distributed training trong Deep Learning.

## B.0.4 PyTorch: Data Parallel

**Mục tiêu:** Minh họa lớp trừu tượng hóa hiện đại trong framework AI.

```
import torch
import torch.nn as nn

model = nn.Linear(10, 2)

if torch.cuda.device_count() > 1:
    model = nn.DataParallel(model)

model = model.cuda()

x = torch.randn(32, 10).cuda()
y = model(x)

print(y.shape)
```

### Giải thích:

Đoạn mã trên cho thấy cách PyTorch cung cấp một giao diện rất ngắn gọn để khai thác nhiều GPU thông qua `nn.DataParallel`. Ở mức sử dụng, người lập trình gần như không phải trực tiếp xử lý các chi tiết truyền thông và đồng bộ.

Tuy nhiên, về bản chất, cơ chế bên dưới vẫn dựa trên các nguyên lý quen thuộc của tính toán song song: chia dữ liệu, thực thi song song trên nhiều thiết bị, và đồng bộ hóa kết quả. Điều này cho thấy các framework AI hiện đại không thay thế các nguyên lý song song cổ điển, mà trừu tượng hóa chúng ở mức cao hơn.

## B.0.5 Nhận xét tổng hợp

Các ví dụ trên cho thấy một chuỗi phát triển khá rõ từ các mô hình song song cổ điển đến các framework AI hiện đại:

- OpenMP minh họa song song trong bộ nhớ chia sẻ
- MPI minh họa song song phân tán dựa trên message passing
- MPI Allreduce cho thấy cơ chế đồng bộ có liên hệ trực tiếp với AI
- PyTorch cung cấp lớp trừu tượng hóa hiện đại cho các cơ chế đó

Như vậy, các framework AI hiện đại không tách rời khỏi lập trình song song truyền thống. Ngược lại, chúng kế thừa và đóng gói các nguyên lý nền tảng như phân chia công việc, truyền thông và đồng bộ để phục vụ cho các bài toán học máy quy mô lớn.

# Phụ lục C

## Ứng dụng AI phổ biến tiêu biểu

Phụ lục này trình bày một số ví dụ ứng dụng AI nhằm minh họa vai trò của các nguyên lý tính toán song song trong các bài toán thực tế. Các ví dụ được trình bày ở mức khái niệm và demo, không đi sâu vào toàn bộ chi tiết triển khai nhằm giữ tính cô đọng của báo cáo.

### C.0.1 Nhân ma trận với kỹ thuật Tiling

**Mục tiêu:** Minh họa cách tối ưu phép toán nền tảng trong AI.

**Mô tả:**

Phép nhân ma trận là phép toán cốt lõi trong hầu hết các mô hình học sâu. Trên GPU, phép toán này được tối ưu bằng kỹ thuật *tiling*, trong đó các ma trận lớn được chia thành các khối nhỏ (tiles) để tận dụng bộ nhớ nhanh như shared memory.

**Ý tưởng chính:**

- Chia ma trận lớn thành các block kích thước nhỏ (ví dụ  $16 \times 16$ )
- Nạp từng block vào shared memory
- Thực hiện phép nhân và cộng dồn kết quả theo từng tile

**Liên hệ song song:**

- Partitioning: chia ma trận thành các tile
- Communication: nạp dữ liệu vào shared memory
- Mapping: ánh xạ mỗi tile vào một thread block GPU

**Ý nghĩa:**

Kỹ thuật tiling cho phép tăng khả năng tái sử dụng dữ liệu, giảm truy cập global memory và cải thiện đáng kể hiệu năng tính toán trên GPU.

### C.0.2 Phát hiện sâu bệnh trên lá táo với YOLO

**Mục tiêu:** Minh họa ứng dụng Computer Vision trong nông nghiệp.

**Mô tả:**

Bài toán phát hiện sâu bệnh trên lá cây có thể được giải bằng các mô hình object detection như YOLO. Mô hình nhận ảnh đầu vào và dự đoán vị trí (bounding box) của vùng bị bệnh.

**Pipeline:**

- Thu thập dữ liệu ảnh lá cây
- Gán nhãn dữ liệu (labeling)
- Huấn luyện mô hình YOLO
- Thực hiện suy luận (inference) trên ảnh mới

**Liên hệ song song:**

- Data Parallelism: huấn luyện trên nhiều GPU
- Tensor Parallelism: xử lý các phép toán convolution lớn
- GPU Acceleration: tăng tốc quá trình inference

**Ý nghĩa:**

Ứng dụng này cho thấy AI có thể hỗ trợ nông nghiệp chính xác, giúp phát hiện sớm bệnh và giảm thiểu thiệt hại.

### C.0.3 Dự báo lũ cho Đồng bằng sông Cửu Long

**Mục tiêu:** Liên hệ với bài toán thực tế tại Việt Nam.

**Mô tả:**

Dự báo lũ là một bài toán chuỗi thời gian phức tạp, phụ thuộc vào nhiều yếu tố như lượng mưa, mực nước và dòng chảy.

Các mô hình có thể sử dụng:

- LSTM

- Transformer
- Các mô hình lai (Hybrid models)

**Liên hệ song song:**

- Data Parallelism: huấn luyện trên nhiều chuỗi dữ liệu
- Distributed Training: tăng tốc huấn luyện mô hình lớn
- HPC Cluster: xử lý dữ liệu khí tượng quy mô lớn

**Ý nghĩa:**

Ứng dụng này có tiềm năng hỗ trợ cảnh báo sớm thiên tai, đặc biệt quan trọng đối với khu vực Đồng bằng sông Cửu Long.

**C.0.4 Nhận xét tổng hợp**

Các ví dụ trên cho thấy:

- Các nguyên lý tính toán song song xuất hiện xuyên suốt từ mức thấp (matrix multiplication) đến mức ứng dụng (computer vision, time series)
- GPU đóng vai trò trung tâm trong việc tăng tốc các bài toán AI
- Các framework AI hiện đại giúp đơn giản hóa triển khai, nhưng vẫn dựa trên các primitive song song như partitioning và synchronization

Do đó, việc hiểu các nguyên lý song song không chỉ mang tính lý thuyết, mà có ý nghĩa trực tiếp trong việc xây dựng các hệ thống AI thực tế.

## Phụ lục D

# Minh họa thực nghiệm trên GPU và liên hệ với tính toán song song

Phụ lục này trình bày các ví dụ thực nghiệm trên GPU nhằm minh họa các nguyên lý tính toán song song trong AI hiện đại, đồng thời kết nối với các khái niệm đã phân tích trong báo cáo.

Các thí nghiệm được thực hiện trên máy tính cá nhân sử dụng GPU NVIDIA RTX 3080 với CUDA, và được lập trình bằng Python thông qua framework PyTorch.

### D.0.1 Môi trường thực nghiệm

Các thí nghiệm được triển khai với cấu hình:

- GPU: NVIDIA RTX 3080
- Framework: PyTorch (CUDA-enabled)
- Ngôn ngữ: Python
- Backend: CUDA / cuDNN

Trong bối cảnh này, song song hóa chủ yếu diễn ra ở cấp độ GPU, thông qua các kernel CUDA và các phép toán tensor được tối ưu hóa.

### D.0.2 Demo 1: Phân loại bệnh lá táo trên GPU

Repo `plantvillage-applergb` được sử dụng để minh họa bài toán phân loại ảnh trong computer vision.

**Quy trình:**

- Tiền xử lý dữ liệu (resize, normalization)
- Huấn luyện mô hình CNN (ResNet18)
- Đánh giá trên tập validation
- Xuất mô hình sang ONNX và TorchScript

### **Từ góc độ song song:**

#### **(1) Song song hóa ở mức tensor**

Các phép toán như convolution và matrix multiplication được thực thi dưới dạng kernel CUDA, cho phép xử lý song song trên số lượng lớn thread GPU.

#### **(2) Batch-level parallelism**

Dữ liệu được tổ chức theo mini-batch, trong đó mỗi batch được xử lý song song trên GPU, tương ứng với một dạng data parallelism ở cấp thiết bị đơn.

#### **(3) Tối ưu bộ nhớ và data movement**

Hiệu năng phụ thuộc mạnh vào việc truyền dữ liệu từ CPU sang GPU và cách tensor được tổ chức trong VRAM.

#### **(4) Mapping lên phần cứng**

Các phép toán tensor được ánh xạ trực tiếp lên GPU, thể hiện rõ bước Mapping trong mô hình PCAM.

## **D.0.3 Demo 2: Dự báo mưa**

Repo `rainfall_prediction` được sử dụng để minh họa bài toán dự báo chuỗi thời gian và dữ liệu không-thời gian.

Mô hình sử dụng các kiến trúc như ConvLSTM, trong đó mỗi bước thời gian yêu cầu xử lý các tensor nhiều chiều.

### **Từ góc độ song song:**

#### **(1) Song song hóa theo không gian và thời gian**

ConvLSTM kết hợp convolution (song song theo không gian) và recurrence (phụ thuộc theo thời gian), tạo ra cấu trúc tính toán phức tạp hơn so với CNN thuần.

#### **(2) Trade-off giữa parallelism và dependency**

Các phép toán convolution có thể song song hóa mạnh, nhưng phụ thuộc theo thời gian trong LSTM giới hạn khả năng song song hóa toàn phần.

### (3) Chi phí bộ nhớ

Dữ liệu không-thời gian có kích thước lớn, làm tăng áp lực lên VRAM và yêu cầu tối ưu layout bộ nhớ.

## D.0.4 Liên hệ với các nguyên lý song song

Hai ví dụ trên cho thấy rằng, ngay cả trong môi trường một GPU, các nguyên lý song song hóa vẫn thể hiện rõ:

- **Partitioning:** chia dữ liệu thành mini-batch
- **Communication:** truyền dữ liệu giữa CPU và GPU
- **Agglomeration:** gom batch để tăng hiệu quả tính toán
- **Mapping:** ánh xạ các phép toán lên GPU

Điều này cho thấy GPU không chỉ là thiết bị tăng tốc, mà là nền tảng thực thi song song cốt lõi của AI hiện đại.

# TÀI LIỆU THAM KHẢO

- [1] D. Narayanan **and others**, “Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM,” *in Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)* 2021.
- [2] Y. Huang, Y. Cheng, A. Bapna **and others**, “GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism,” *Advances in Neural Information Processing Systems*, 2019.
- [3] S. Rajbhandari, J. Rasley, O. Ruwase **and** Y. He, “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *in Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)* 2020.
- [4] I. Foster, *Designing and Building Parallel Programs*. Addison-Wesley, 1995. **url**: <https://www.mcs.anl.gov/~itf/dbpp/text/book.html>
- [5] A. Sergeev **and** M. D. Balso, “Horovod: Fast and Easy Distributed Deep Learning in TensorFlow,” *in arXiv preprint arXiv:1802.05799* 2018.